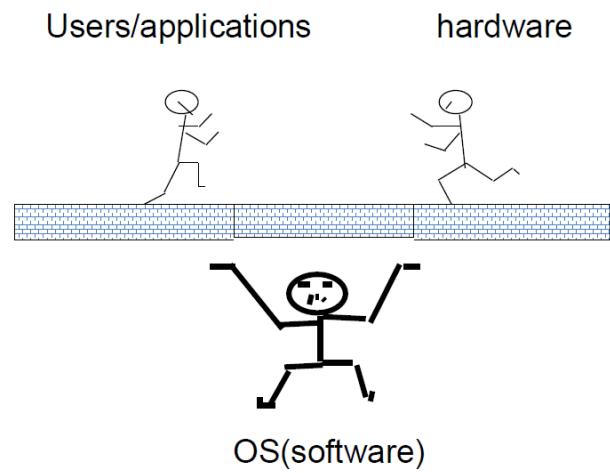


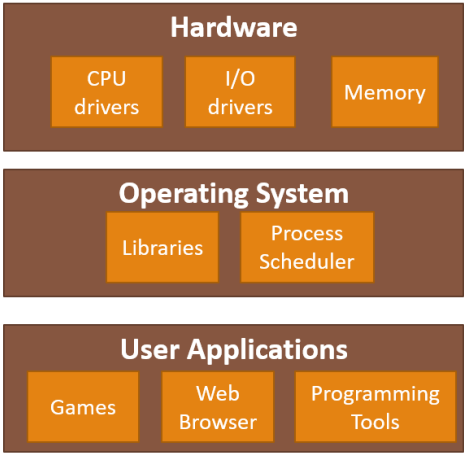
Background on Operating Systems

Operating Systems Bridge the Gap

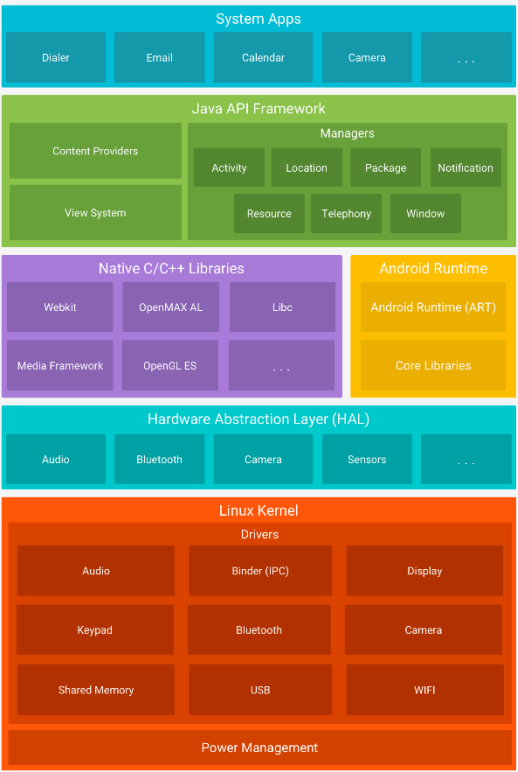


- A computer system can be summarized into three components:
- Hardware components – physical components and drivers.
 - OS – coordinates use of hardware with user applications
 - Applications – typical applications used by a computer user

- The primary tasks of an OS:
- A bridge/API to hardware resources.
 - Resource allocator – allocates resources such as CPU and memory to an application. Decides how to share resources among other user programs.
 - Control program – controls behavior of user programs to avoid improper use of computer resources (CPU and memory hoggers, programs with infinite loops, etc).
 - Security - Must protect application’s data from one another (unless designed to share the data). Allocate resources fairly and efficiently. Settles conflicting requests for resources. Prevent errors and improper use of the hardware
 - Improvisation – find a way to improvise when hardware resources are scarce. Provides an illusion of dedicated machine with infinite memory and processors.

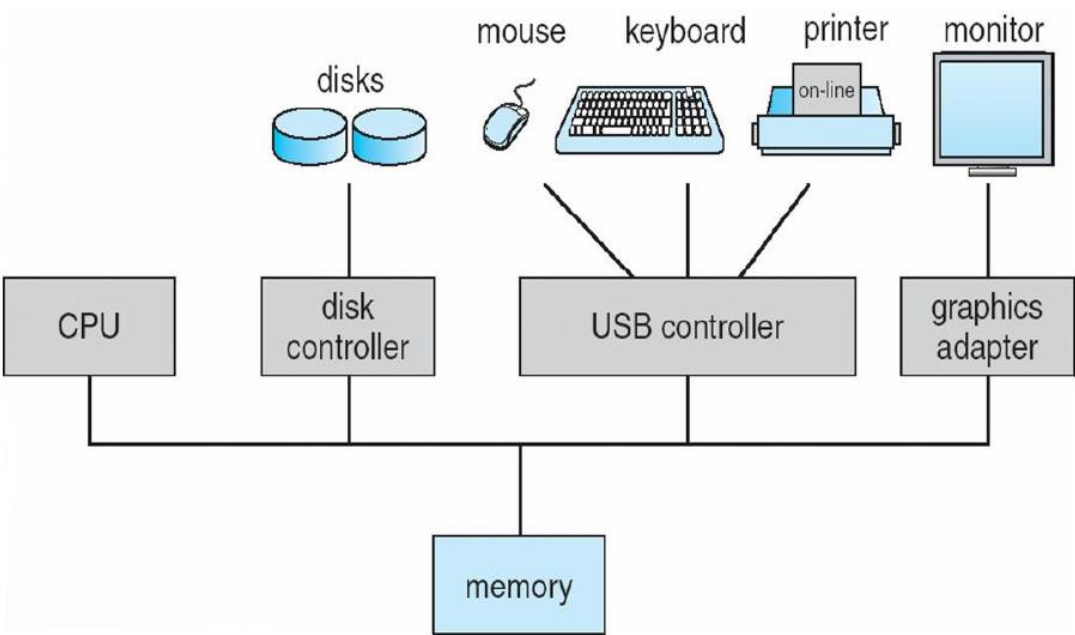


Architecture of android



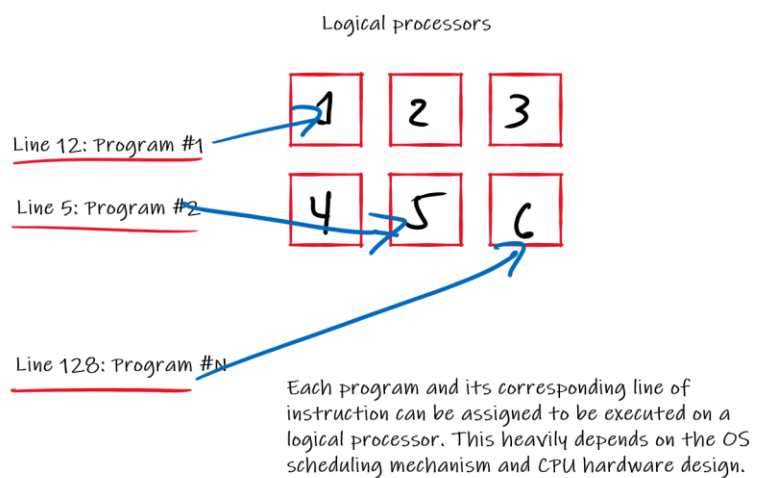
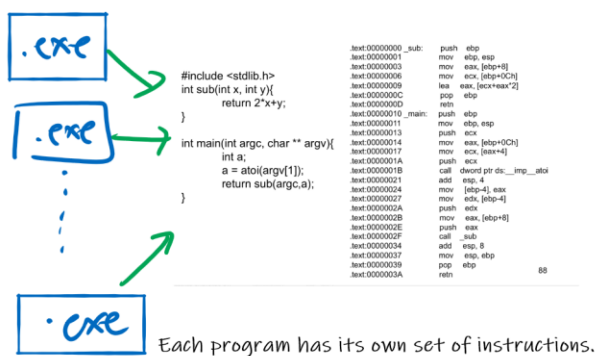
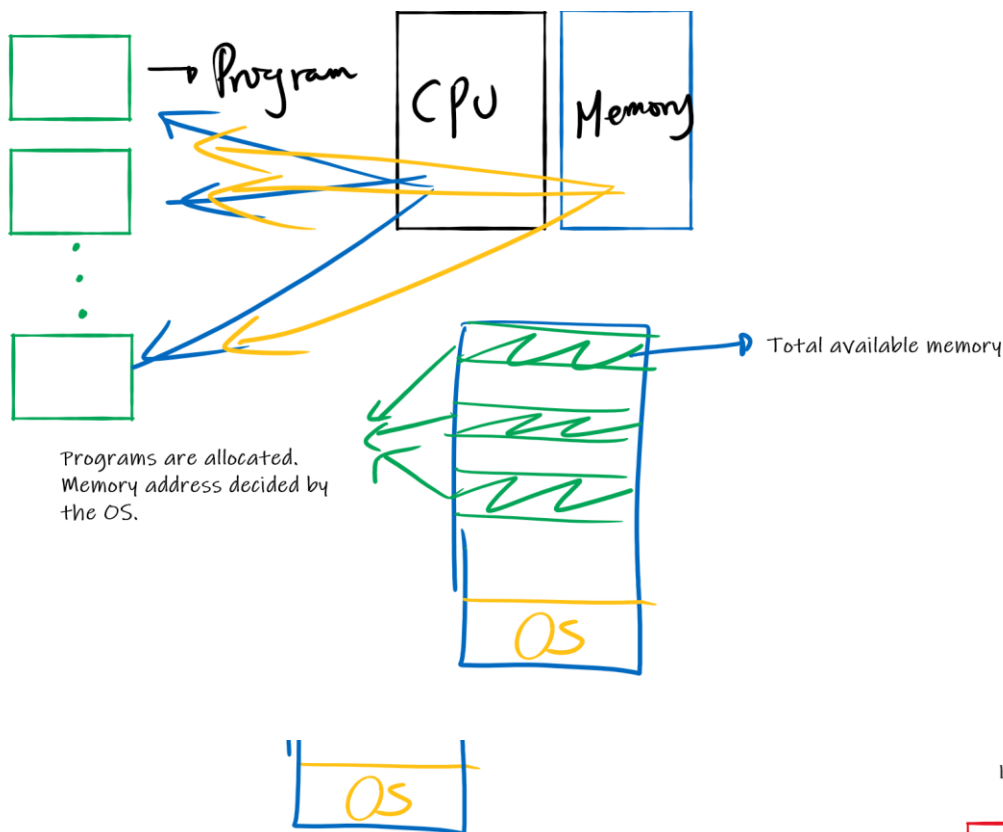
An example of primary tasks in action:

A bridge/API to hardware resources



- How do we display something on-screen? → display interfacing, screen buffers, refresh rates.
- How do we access input devices? → getch(), keyin(), std::in API calls.
- How do we access storage? How do we organize our files in storage? → file directory systems implementation, user access permissions.

Resource allocator

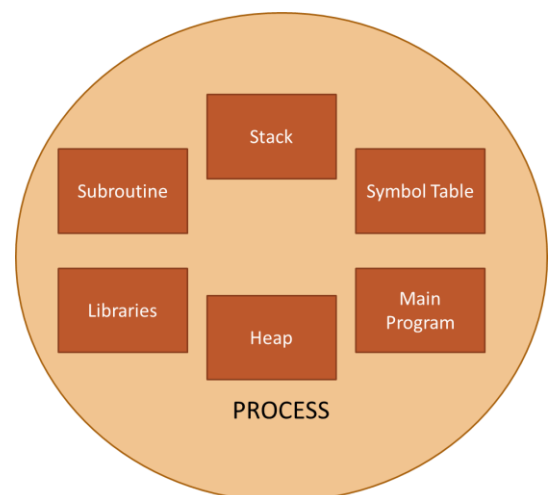


Control program

The OS ensures that all applications receive a fair number of resources. Each program has components where allocations are already pre-defined.

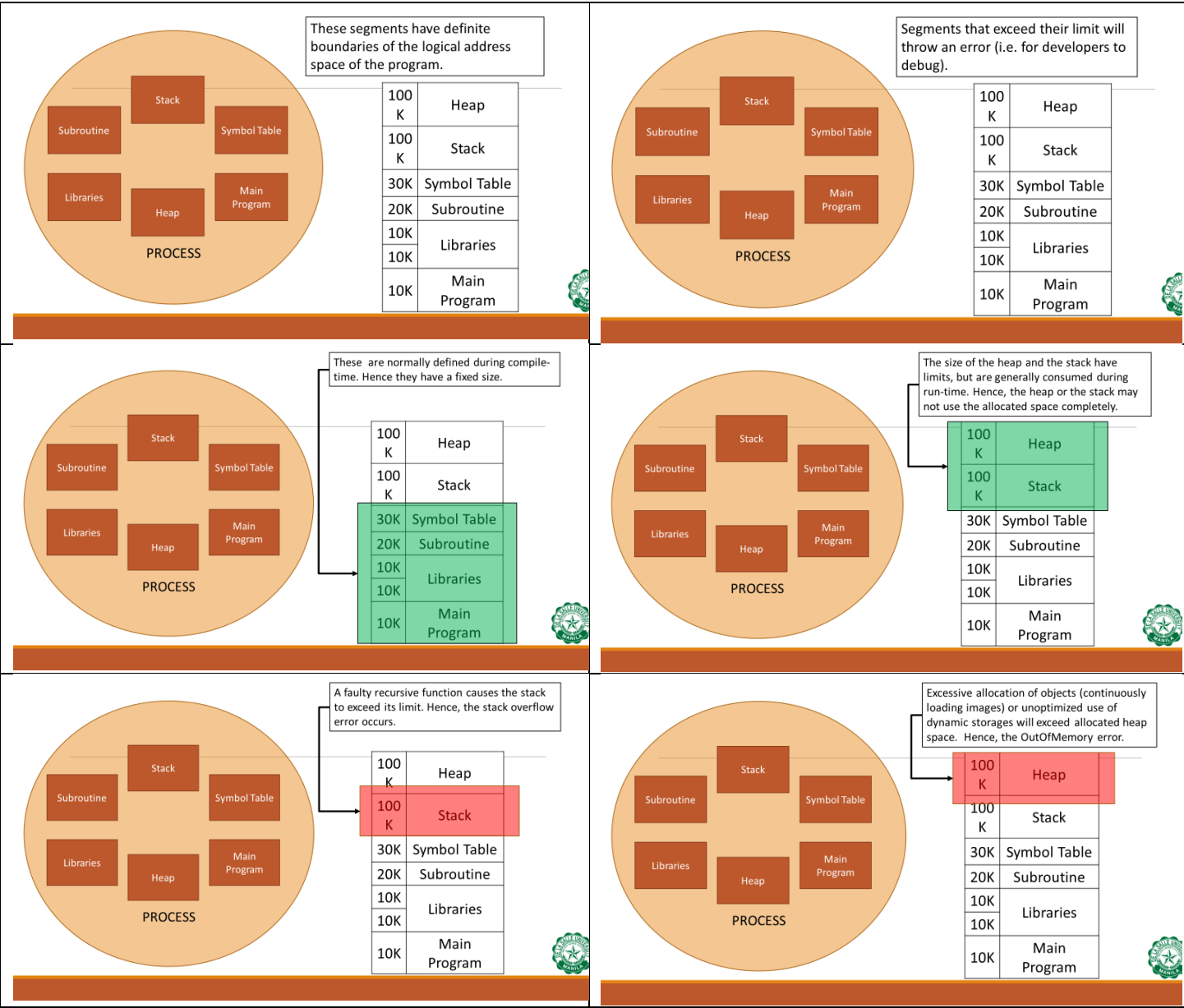
Components of a Program

- Main Program – points to the main function
- Subroutine – contains all functions declared in code
- Libraries – external functions or dependencies required
- Symbol table – holds variables
- Stack – holds temporary variables for function calls. Also stores the return address of a given function call.
- Heap – memory for dynamically allocated objects



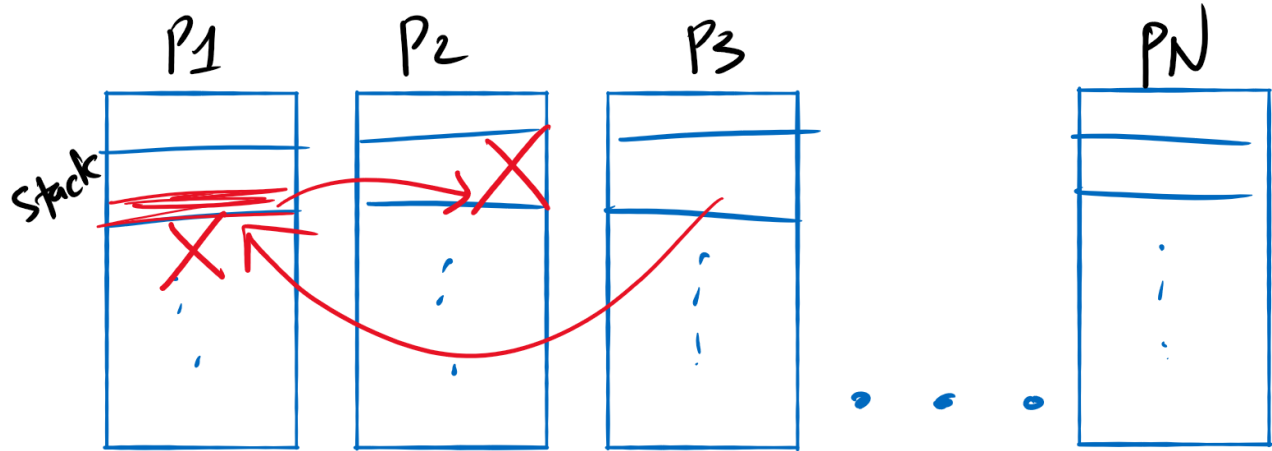
ACTIVITY:

1. Provide a scenario that causes stack to overflow.
2. Provide a scenario that causes the heap to overflow.

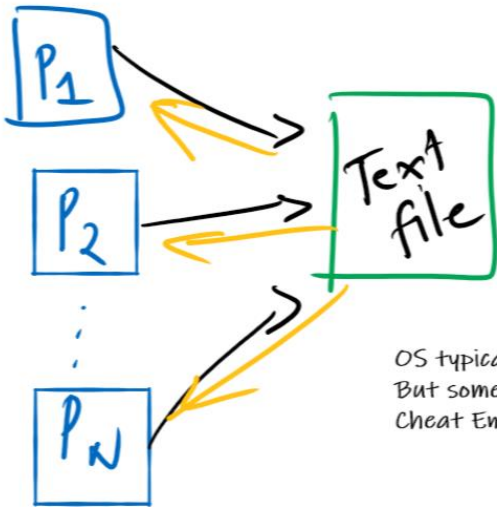


The OS pre-allocates memory on each application. Each application has a certain boundary on their components. Overstepping these boundaries throw errors.

Security

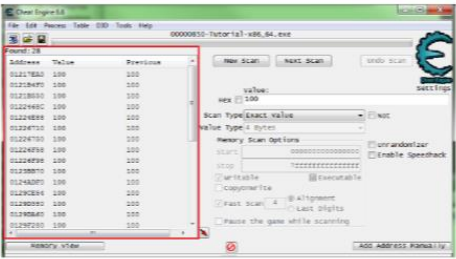


Each process has its own allocation and cannot be accessed by any other process.



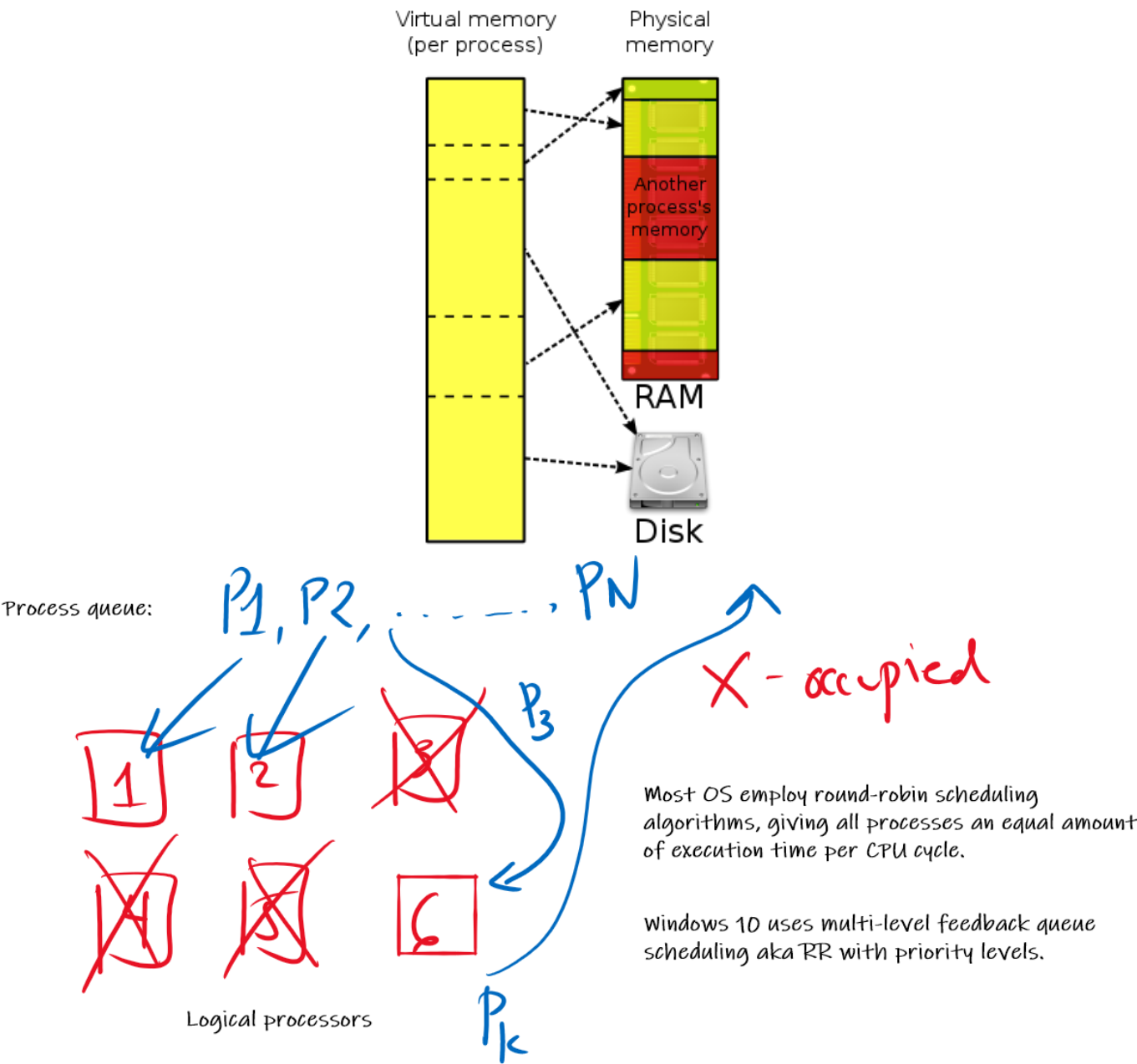
If processes need to share information, each process must have instructions on writing/reading a shared file.

OS typically deny any malicious memory accesses. But some tools exist to circumvent this (e.g. Cheat Engine).



Improvisation

In memory management, all OS perform a technique called **virtual memory**, where the OS uses secondary storages (such as HDD/SDD) as main memory whenever RAM is used up. This gives the illusion of a very large main memory.



Why do we need to study operating systems?

- First, we can't create an OS on-par with most commercial OS nowadays → most mainstream OS (Windows/Mac/Linux) already have systems built on top of one another, some of which are older than us! (OS Dev Wiki – beginner mistake hard truth)
- An exposure to systems-level development, which may not be taught from other CS courses. Answers some of the questions like, “who creates and maintains my programming language?”, or “who's behind and how does my program work with different OSes”
- Most of the algorithms in OS, can actually be applied to other applications. E.g. real-time and interactive applications. CPU scheduling → real-time scheduling and distribution of items in a game.
- Prerequisite knowledge for designing concurrent and distributed systems → E.g. Netflix video streaming. Different servers work together for you to watch your favorite movie in Netflix → video compression/decompression, video file indexing, media databases. All of these would require coordination algorithms, which are rooted in OS concepts.
- An opportunity to study C++! → It is a powerful language designed for systems development and game development, and you could do and emulate a lot of OS operations, than other languages.

Why is it challenging to study operating systems?

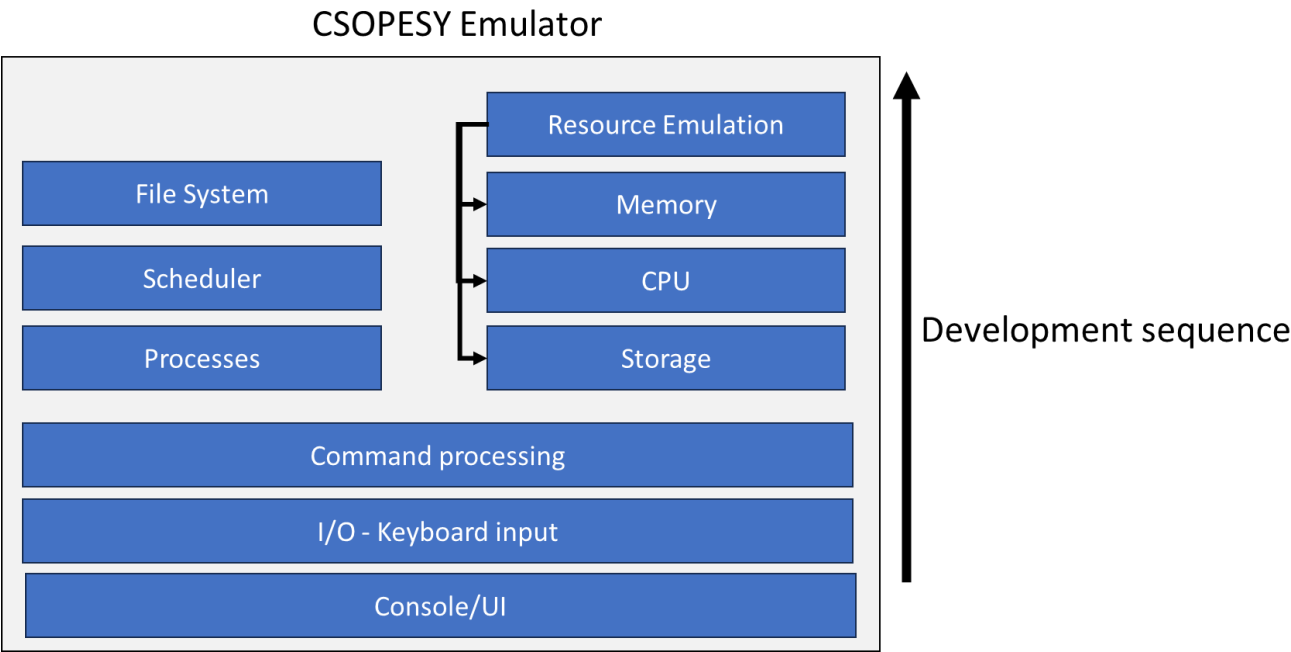
- If we try to create an OS from the ground up, all the syntax-checking, IDEs, and pre-built APIs are not available – we have to code each and every one of them. E.g., Your own stack function, memory allocation function, and threading function.
 - To make things a little bit easier for CSOPESY, we're going to use C++ and its associated system and threading libraries. 😊
- It is difficult to debug an OS as there could be many possible scenarios, combinations of which could be happening. A single test run will differ from another test run. E.g. different sequences of concurrent processes boot up at every run.
 - We also need to code our debugging tools – a memory visualizer, a meaningful format for our log messages, verbosity, etc.
- The program is purely text-based and algorithmic in nature
 - No punchy visualizations or UI. Challenging for developers who are looking for visual elements for evaluating their program.
 - Good UI/UX design should only come after the foundation of the OS is made. E.g., the kernel, the CLI, the scheduler, the memory allocator, etc.

Background

- What will you work on for most of the term – an OS emulator.
- Features of the CSOPESY OS emulator
- An overview of its proposed design of systems to get you started.
- Preview of assessments for your problem sets.

The CSOPESY emulator

- An emulator of different OS concepts – we will code the following important features: I/O interface, keyboard input, drawing/refreshing the screen, file directory system, process scheduling, and memory management.



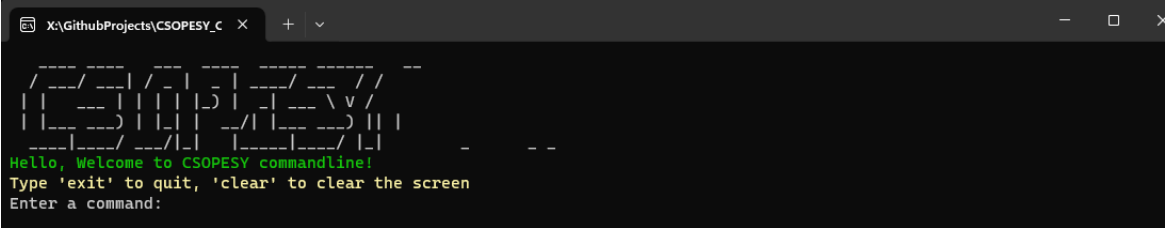
A typical sequence of how an OS is loaded and run

```
682 #include <iostream>
683
684 // Function prototypes
685 void bootstrap();
686 void initializeKernel();
687 void startSystemServices();
688 void enterMainLoop();
689 void shutdownAndCleanup();
690
691 // Define the main function
692 int main() {
693     // Step 1: Bootstrapping
694     bootstrap();
695
696     // Step 2: Kernel Initialization
697     initializeKernel();
698
699     // Step 3: Start System Services
700     startSystemServices();
701
702     // Example: Print a message indicating successful boot-up
703     std::cout << "Operating System Booted Successfully!\n";
704
705     // Step 4: Enter Main Loop
706     enterMainLoop();
707
708     // Step 5: Shutdown and Cleanup
709     shutdownAndCleanup();
710
711     // Note: The OS typically never reaches this point
712     return 0;
713 }
```

1. Bootstrapping:
 - Perform low-level initialization (hardware setup, memory initialization, etc.)
 - Load the kernel into memory and start executing it
 - Example: Perform hardware initialization
 2. Kernel initialization
 - Initialize data structures (process table, file system, etc.)
 - Set up interrupt handlers and device drivers
 - Initialize memory management and scheduling algorithms
 - Example: Initialize process table and memory manager (e.g. memory allocation algorithm, demand paging)
 3. Start system services
 - Start essential system services (file system, networking, etc.)
 - Launch system daemons and background processes
 - Example: Start file system service and network service
 - Observation: These are processes that are persistent in the OS – always alive/periodically invoking functions
 4. Enter main loop
 - Continuously handle interrupts and system calls
 - Dispatch user processes and manage their execution
 - Handle user input and manage I/O operations
 - Example: Enter an infinite loop to handle system events
- ```
737 // Step 4: Enter Main Loop
738 void enterMainLoop() {
739 // Example: Enter an infinite loop to simulate OS operation
740 while (<user hasn't initiated shutdown>) {
741 // Continuously handle interrupts and system calls
742 // Dispatch user processes and manage their execution
743 // Handle user input and manage I/O operations
744 // Example: Enter an infinite loop to handle system events
745 }
746 }
```
- 5. Shutdown and cleanup
    - Gracefully terminate running processes and services
    - Clean up allocated resources and release memory
    - Halt or reboot the system
    - Example: Clean up resources and prepare for shutdown

**A walkthrough of its features**

A command-line interface type of application (CLI) → akin to Windows Powershell, Bash, DOS, and other related command-line applications.

- The main menu screen.
- 
- The screenshot shows a terminal window with a title bar 'X:\GithubProjects\CSOPESY\_C'. It displays a large ASCII art logo that resembles a stylized 'C' or a computer monitor. Below the logo, the text reads: 'Hello, Welcome to CSOPESY cmdline!', 'Type \'exit\' to quit, \'clear\' to clear the screen', and 'Enter a command:'.
- Main menu interface accepts commands that transition the user to other console windows. E.g., file directory viewer, marquee console, process scheduler, memory emulator.
  - Drawing of new console layout is done in the same window. No pop-up.
  - Marquee console
    - This is simply a demonstration of creating an immediate-mode application → consistently listens to user inputs while refreshing the screen per frame.
    - A preview of cooperative multitasking that will be needed on succeeding features.



The screenshot shows a Windows command prompt window with the title bar "X:\GithubProjects\CSOPESY\_C". The window contains the following text:

```

* Displaying a marquee console! *

Hello world in marquee!

Enter a command for MARQUEE_CONSOLE: This is a sample|
```

The text "Hello world in marquee!" is displayed in the center of the window. The prompt "Enter a command for MARQUEE\_CONSOLE: This is a sample|" is at the bottom, with a cursor at the end.

- File directory manager

```
Microsoft Visual Studio Debu x + - -
What's the dir? root
What's the dir? root/folder_8/folder_4/folder_10/folder_7/folder_1/f_1.txt/WcysyYcqpeViKeFFmZnImkksvswsrenZKyCxFLSGyps
Fadpo
What's the dir? folder_7/folder_1/folder_4/f_259.txt/vDMJYJnoAqTCyRbZzcyrjBUsmDdVcMJoeKrLzvnGdhSftCzzzL
What's the dir? folder_1/folder_3/folder_2/f_989.txt/sDleuSPcKcspafIDuXDZecPwKnlQWycQzosaJyKYnwNoIfBh
What's the dir? f_571.txt/DTMqOLMmgJqaVQLNRDFDXJGdzQwmoeNQwJUBTsdATNUszPUOoq
What's the dir? folder_5/folder_3/folder_2/f_308.txt/rDZiQGqLcAvoFGpPbqowmouMYUKnJvmpSSdeqknCZngYOfcvZ
What's the dir? f_38.txt/PSVKnhKWJOPMmLebKsUCvzJlqynwCmbVbhRMLoWpJBWyyvtWtGo
What's the dir? f_231.txt/iWLPfIByanraAHOAWOMXZorZGVKtIaZLzYnKmmbcwrjPDZQsgg
What's the dir? f_793.txt/XWntZHgjtIiJXVhJQTUDOGjCmgHcmTehQlyetXIEBQKdli
What's the dir? folder_2/folder_7/f_18.txt/mdcASZsgOVsDkjhGHCMMbMxmHuyfyggaJzKnlZnayXkQqoyz
What's the dir? folder_9/f_171.txt/WoTzqbLhMRfzfGYKZmucwPdvJvCtVfAmGLcJmTJrcJIVHSYt
What's the dir? folder_5/folder_10/folder_8/f_294.txt/TGiYvtyKLPnPhBwYostmWmsJrqPrxNtmLQyLXmnjGCRbIoYhDo
What's the dir? folder_9/folder_7/f_759.txt/gVUZJxhrtqQMScxIahhAheIkyTAGZEICxLgIoKdrIwSrHvM
What's the dir? folder_1/folder_8/folder_5/f_308.txt/mthYhLOxnwuEEosIUUPsZLXkyFrLlOMgvpnsEvMIdmrPBHEpWG
What's the dir? folder_10/folder_6/f_912.txt/dvOgXWgZTRaVCLcJLrtEBmSQVNTLIVRAPoLPLSBqAoFZCMAuX
What's the dir? folder_4/folder_2/folder_10/f_773.txt/vJgpUrgFsyIDvcAThmwxvzrJqKXZnZtJjauDeKLPwYthfw
What's the dir? folder_4/folder_1/f_808.txt/yeXffNgBjPjERrtJzonAPDmLWycoPKyJaeHvZNMtrfEGjFcnSn
What's the dir? folder_8/folder_3/folder_4/f_867.txt/mkmhUDtXFYxpRHOLDjupdjYHtLftlgEicLwumLABkmsOwNvHqrm
What's the dir? f_897.txt/gYbWFLMGQSFajUQKbsBhpNbnZQavgyAwnHayaIamghpzhMmCnT
What's the dir? folder_6/f_941.txt/blbWwEaqWorNerTVgx8VWIXisjwvOIGFrtZSmunnJxmJmvgqFzL
What's the dir? folder_2/folder_4/folder_8/f_959.txt/ZJTgyavBABfNJKBJxtrDhPBcnLNKSHJSkzyBxEjCqJjowsvLf
What's the dir? folder_7/folder_8/folder_4/folder_2/f_22.txt/kJmjQwIPuoqhImFvuZwYvIjgfLkjdUhsJaFbtLKMfqrMyJ
What's the dir? folder_9/folder_5/f_535.txt/cnKLPSmPznLSDRPUfxrIOwdSalfcjXARhbluJxhdLKLRLGLqWP
What's the dir? folder_9/folder_1/folder_7/f_615.txt/nPEEjnhHYICpkhJNayVHYfdyVLQxZLmKmsgnvjeqfcsFhTqws
What's the dir? folder_2/folder_8/folder_8/f_657.txt/vyzFFxexNwPFNckWariaQXjSEAMitCvHeoVaeBjQghZJCvic
What's the dir? folder_6/folder_3/folder_4/f_752.txt/uDcNzgFMhYhHKZyZWrgIzgoPKQbXAdmxrImothqQaoEadIS
What's the dir? folder_4/folder_9/folder_8/f_917.txt/IsSrclLcOCCWUJZrJfLvgdxYqCdcDrYcCclxrvartGVmihHaE
```

- Akin to file directory managers of most OS.
- Supports standard file directory commands, like **mkdir**, **cd**, **createdir**, **search**, **dirlist**, etc.
- All files are represented as a **string of bytes**, all files are bundled together into a single **.csopesy file**.
- A file directory system reads the .csopesy file, parsing and interpreting the multi-level directory structure.
- Certain files may be required by certain processes → depending on configuration.

- Scheduling

```

25
26
27
28
29
30 Welcome to CSOPESY Emulator!
31
32 Developers:
33 Del Gallego, Neil Patrick
34
35 Last updated: 01-18-2024
36
37 root:\> screen -ls
38 CPU utilization: 100%
39 Cores used: 4
40 Cores available: 0
41
42 -----
43 Running processes:
44 process05 (01/18/2024 09:15:22AM) Core: 0
45 process06 (01/18/2024 09:17:22AM) Core: 1
46 process07 (01/18/2024 09:17:45AM) Core: 2
47 process08 (01/18/2024 09:18:58AM) Core: 3
48
49 Finished processes:
50 process01 (01/18/2024 09:00:21AM) Finished
51 process02 (01/18/2024 09:00:22AM) Finished
52 process03 (01/18/2024 09:00:42AM) Finished
53 process04 (01/18/2024 09:00:53AM) Finished
54 -----
55
56 root:\> report-util
57 root:\> Report generated at C:/csopesy-log.txt

```

- A scheduling simulation → simulates various CPU scheduling algorithms, given N processes and X CPU cores. Note that CPU cores are simulated/declared during compile-time.
- An arbitrary number of processes are scheduled. Each process contains Z commands → commands could either be a simple print or an I/O operation such as searching for a file or a request for a driver/hardware.
- Can display a summary of the order of processes finished/still executing.
- Memory management
  - Simulation that combines memory management + scheduling → using existing CPU schedulers implemented, processes only execute if sufficient memory is available.
  - Has a command that displays memory utilization and running processes. E.g. `nvidia-smi`

```

X:\GithubProjects\CSOPESY_C X + v

* Displaying a memory management console!
* Using scheduler : MEMORY_CONSOLE Using memory emulation: FlatMemory

Enter command: csopesy-smi
-----+-----
CSOPESY-SMI V001.000.00 Driver Version: 999.999.99 CUDA Version: 11.2
-----+-----
Memory Usage		Memory Utilization
00000 MB / 240000 MB		100%
+-----+-----+		
Processes:		
CPU # PID Process Name Memory Usage		
=====		
Enter command:

```

- Processes have a pre-defined amount of memory needed to execute. The emulator allocates the needed memory, if available.
- Allows user input-driven creation of processes on the fly → while having a predefined list of running processes, the user can add new ones. All processes must finished eventually → CPU and memory utilization should show 0%.
- Allows real-time viewing of CPU and memory utilization, while still typing commands.

**Sample assessments for each feature**

- This is to help set expectations on how your machine project would be graded.
- Typical approach: Online quiz system → submit .mp4 or .png files as proof of output, using your running OS emulator.
- Online quiz system is in time-pressure format (e.g. 2 hours or 1 hour) → this is to avoid drastic changes to your code, hacking, band-aid fixing, and frequent recompiling just to meet the expected output → promotes pro-active QA/testing of your app.
- The test cases could be given earlier or when the quiz is started.

**Marquee console and command-line interface**

| Test                                                                                                   | Expected output                                                                                                                          |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Typing “the quick brown fox jumped over the lazy dog” with pre-defined animation speed for the marquee | The marquee should behave/move in accordance with the set animation speed while keeping the keyboard polling active as much as possible. |
| Ability to recognize placeholder commands for future use: scheduling, filesystem, memory               | Commands are recognized properly, while the marquee is still animating.                                                                  |

Needed programming concepts: I/O and display interface, parsing commands, threads.

**Scheduling**

- Each process is attached to a screen and the overall application behaves like a process multiplexer.
- Please refer to a general Linux/Windows powershell/Windows command line. This serves as a strong reference for the design of your command-line interface.
- For the process multiplexer, refer to the Linux “screen” command on its behavior: <https://www.geeksforgeeks.org/screen-command-in-linux-with-examples/>

| Test                                         | Expected output                                                                                                               |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| The teacher will provide the screen commands | All screen commands and processes associated with it must execute/finished execution using the intended scheduling algorithm. |

Needed programming concepts: reading/writing files (for parsing the test case input), CPU scheduling, threads.



Memory management

- There must be a mechanism to visualize and debug memory. The user can use either “process-smi,” which provides a high-level overview of available/used memory, or “vmstat,” which provides fine-grained memory details.
- The “process-smi” is similar to the nvidia-smi command that prints a summarized view of the memory allocation and utilization of the processor (CPU for your program / GPU for nvidia-smi).

| Test                                                                               | Expected output                                                                                   |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Given a batch of processes, provide periodical memory snapshots at fixed intervals | The memory snapshots should report the overall memory usage, free memory, fragmented memory, etc. |
| Simulate a low memory environment                                                  | The memory report should show high thrashing activity and high page swaps.                        |

Needed programming concepts: I/O and display interface, parsing commands, threads, CPU scheduling, cooperative multitasking, deadlock handling, memory management.

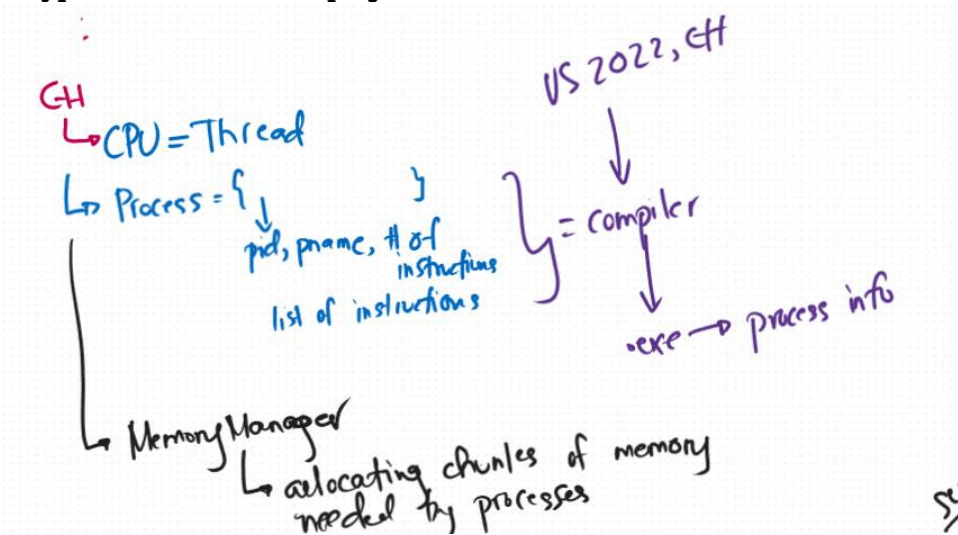
File directory manager

- The teacher sends a sample .csopesy file → the student performs the following tests

| Test                              | Expected output                                                            |
|-----------------------------------|----------------------------------------------------------------------------|
| Searching for a specific file     | Outputs “file not found” or “file found”                                   |
| Searching for a keyword           | Outputs lists of files that met the search keyword                         |
| Creation of directories and files | Able to output the newly created directory path, as well as the file path. |

Needed programming concepts: file systems, reading/writing files, parsing commands.

A typical Visual Studio project for CSOPESY emulator



- A CPU could be represented by an inherited std::thread class.
- A process could be represented as another class containing important information such as: process id, name, number of instructions, and types of instructions.
- The memory allocator could be created as another class where various memory allocation algorithms will be implemented.

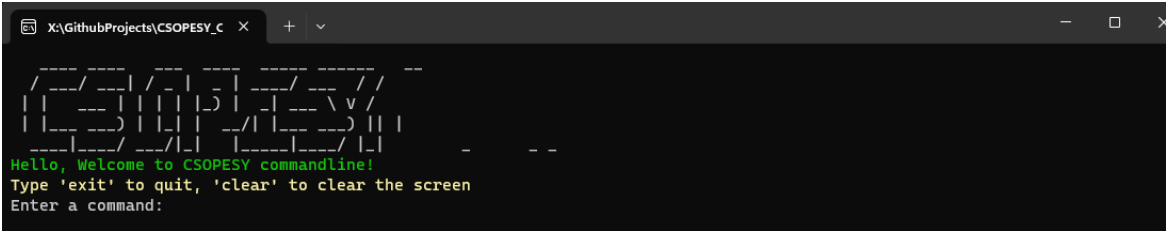
Things to discuss

Discuss hands-on activity: CSOPESY-Emulator tour.



Activities

#1: Setting up your own CSOPESY project



- 
- Create your first-ever **main menu** console. Use only the C++ built-in libraries.
- Provide your ASCII text header “CSOPESY” or a name for your command line emulator.
- Set up template code that accepts the following commands: **marquee**, **screen**, **process-smi**, **nvidia-smi**, **clear**, **exit**. Simply print “X command recognized. Doing something.”, where X is the command recognized.
- For the clear command, once recognized, the screen clears, reprinting the header texts.
- For the exit command, the application/CLI immediately closes.
- Be prepared to show your design implementation when called.

#2 Written: Mapping specific OS functions to the general steps

Identify the step in the bootstrapping process for each of the following 35 functions of a commercial OS:

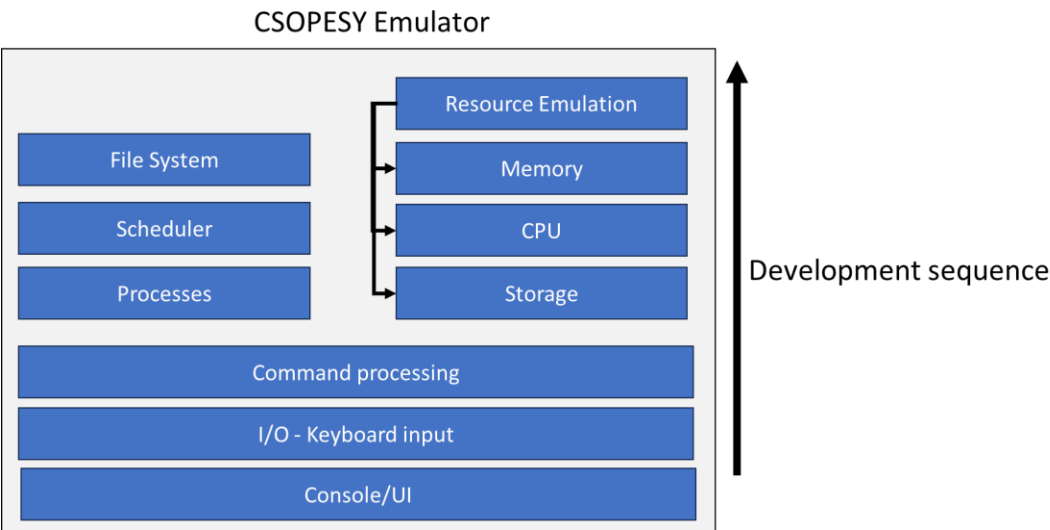
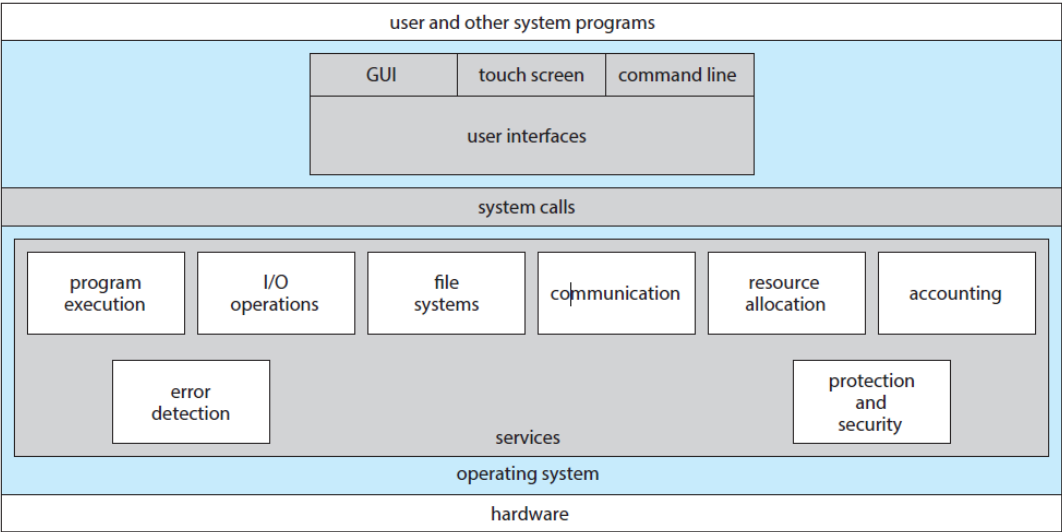
1. Initializing the hardware abstraction layer (HAL)
2. Setting up the Global Descriptor Table (GDT)
3. Enabling the Floating Point Unit (FPU)
4. Loading the initial RAM disk (initrd)
5. Configuring the Interrupt Descriptor Table (IDT)
6. Initializing the Advanced Configuration and Power Interface (ACPI)
7. Enabling virtual memory and setting up page tables
8. Initializing the Programmable Interval Timer (PIT)
9. Setting up the boot loader environment
10. Starting the initial process scheduler
11. Configuring the Memory Management Unit (MMU)
12. Loading kernel modules and drivers
13. Initializing the System Management Mode (SMM)
14. Enabling symmetric multiprocessing (SMP) support
15. Initializing the Real-Time Clock (RTC)
16. Configuring the Unified Extensible Firmware Interface (UEFI)
17. Setting up kernel command line parameters
18. Configuring input/output ports and devices
19. Starting the initial RAM disk filesystem
20. Initializing the system logger
21. Setting up the Real Mode Interrupt Vector Table (IVT)
22. Setting up the initial file system
23. Starting the graphical user interface (GUI)
24. Configuring network interfaces and protocols
25. Initializing user authentication mechanisms
26. Loading and initializing system libraries
27. Setting up inter-process communication (IPC) mechanisms
28. Configuring power management features
29. Initializing system time and date settings
30. Starting system monitoring and performance analysis tool
31. Saving system configuration settings to persistent storage
32. Flushing system buffers and caches
33. Terminating user processes and services gracefully
34. Releasing allocated system resources and memory
35. Closing open file handles and network connections



Background

- To create our emulator from scratch, we need to borrow fundamental OS structures → serve as templates and building blocks for our OS emulator.

Operating system services



- We show a side-by-side comparison of a typical view of commercial OS services available, and what services we need to develop/emulate.

User interface

- Refers to display interface and reading of peripheral inputs → keyboard/mouse? Touch screen?
- Early OS started with command-line interfaces (CLI), which uses text commands and keyboard for entering them → what our OS emulator will look like.

```
1. root@r6181-d5-us01:~ (ssh)
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52, 3 users, load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem Size Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
50G 19G 28G 41% /
tmpfs 127G 520K 127G 1% /dev/shm
/dev/sda1 477M 71M 381M 16% /boot
/dev/dssd0000 1.0T 480G 545G 47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
12T 5.7T 6.4T 47% /mnt/orangefs
/dev/gpfs-test 23T 1.1T 22T 5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root 97653 11.2 6.6 42665344 17520636 ? S<ll Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root 69849 6.6 0.0 0 0 ? S Jul12 181:54 [vpthread-1-1]
root 69850 6.4 0.0 0 0 ? S Jul12 177:42 [vpthread-1-2]
root 3829 3.0 0.0 0 0 ? S Jun27 730:04 [rp_thread 7:0]
root 3826 3.0 0.0 0 0 ? S Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun 3 2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```

```
searsg@login:~
GNU nano 2.3.1 File: pytest.sh Modified

#!/bin/bash

##This is called the SHEBANG. This tells the shell to interpret and run the SLURM
##script using the bash (Bourne-again shell) shell.

##SLURM Commands using #SBATCH --[options]

#SBATCH --job-name demoTest ##give the job a name
#SBATCH --output demo.out ##creates a file to store the job output
#SBATCH --mail-user searsg@uhd.edu ##sends email to user's email for start/complete of job
#SBATCH --mail-type BEGIN ##creates an event type to be mailed to user when event occurs
#SBATCH --mail-type END ##creates an event type to be mailed to user when event occurs
#SBATCH --error test.e.txt ##creates a file to store any error output

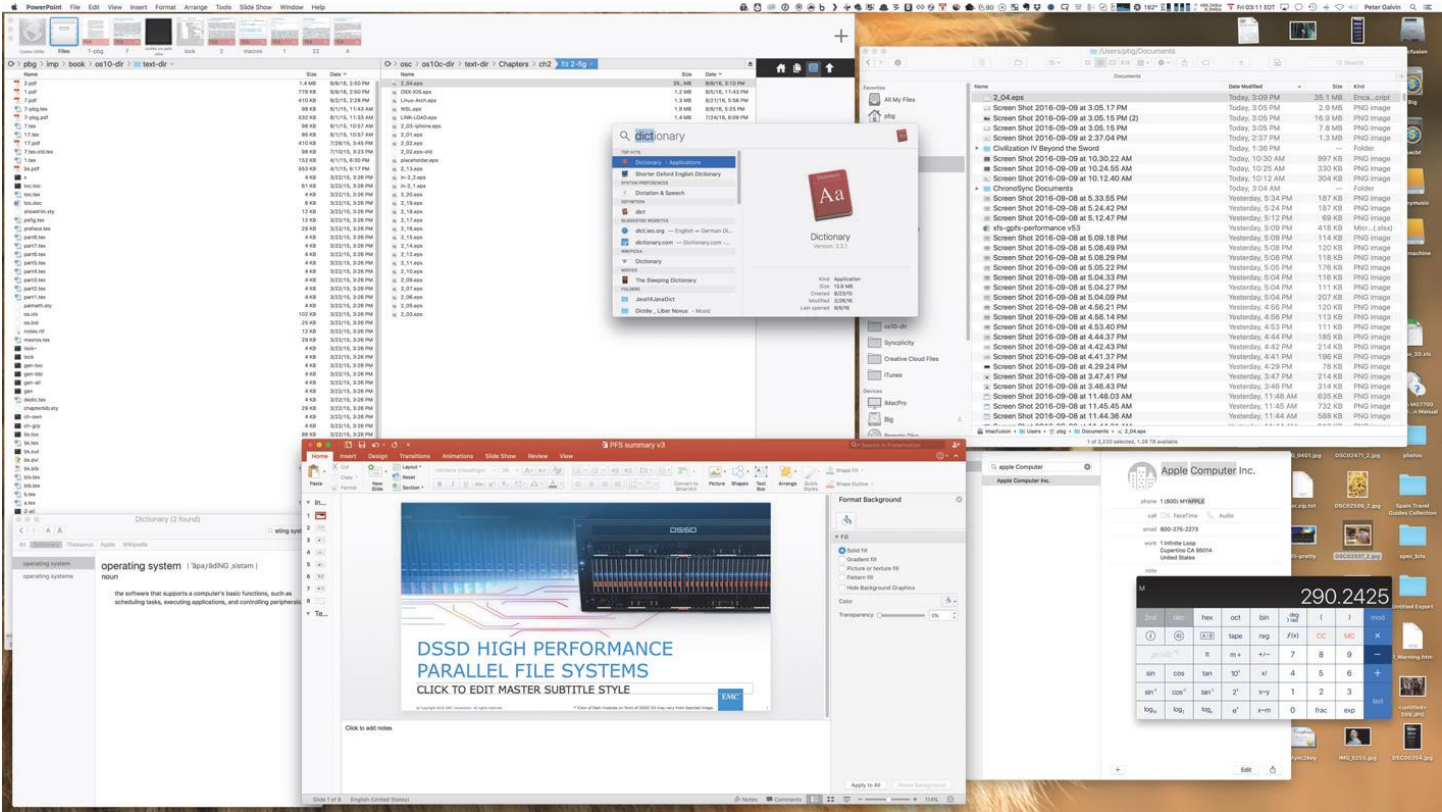
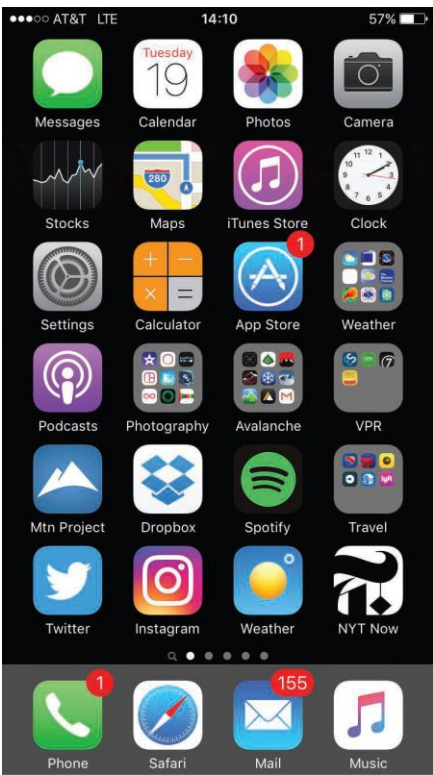
##Load the modules needed for the job to run
module load python3

##List the executables commands (or a string of them)
##Example for code written in Python

python3 pytest.py 90

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

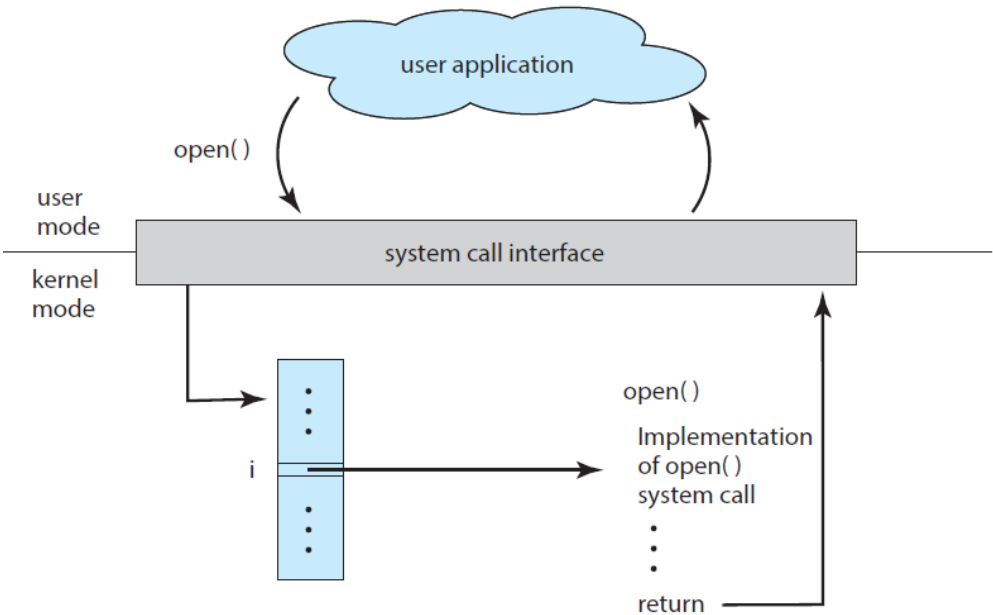
Other interfaces:



Since we will be pursuing a **command-line-interface style emulator**, essential features are:

- Command interpreter
  - Most operating systems, including Linux, UNIX, and Windows, treat the command interpreter as a special program that is running when a process is initiated or when a user first logs on (on interactive systems).
  - Virtual machines that could be rented on the cloud (e.g. Google VM) merely provides a Linux shell with Python and ML libraries pre-installed.
  - Developing a command interpreter require processing of commands received from keyboard input.
    - Maintains a list of commands recognizable: mkdir, cd, tmux, etc.
    - Tokenization of command. E.g. cd “root/example/folder1/subfolder1” → means of splitting the String into tokens using “/” as the separator → checking and validating each directory as the pointer moves to “subfolder1”
- System calls
  - Are libraries/interfaces developed that exposes services in an operating system. E.g. print function, read/write of files.

- Also refers to system APIs. Several programming languages have system APIs that are automatically included/imported in every program. E.g. print/println, class/object management.



- The following are example system calls:

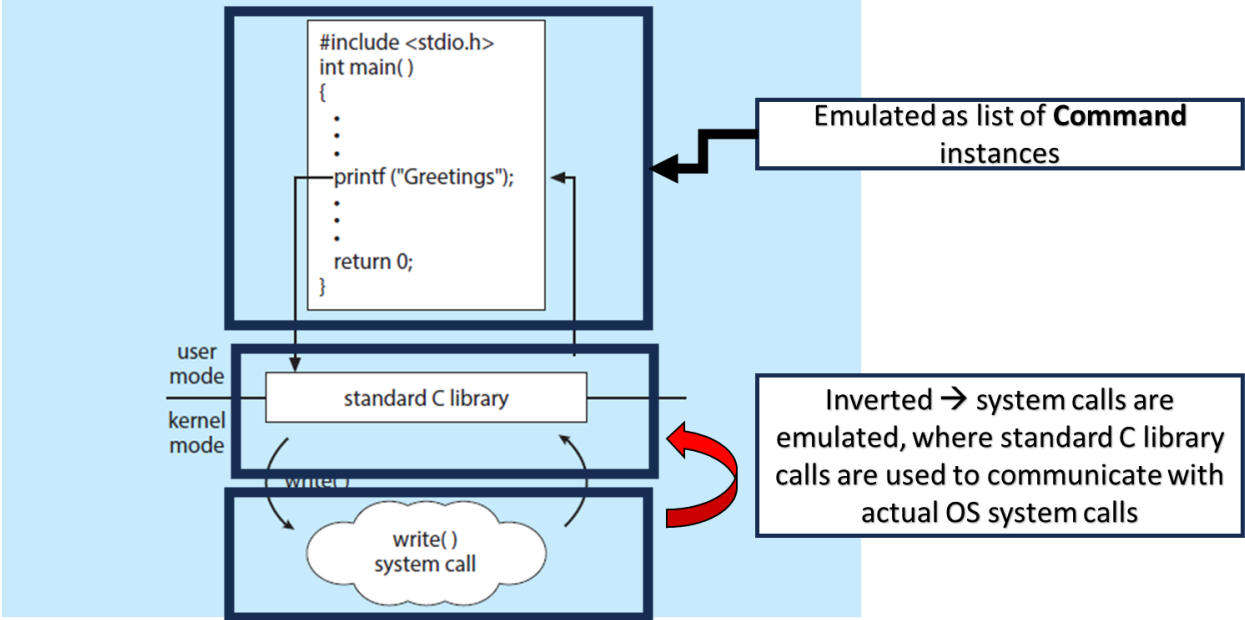
- Process control
  - create process, terminate process
  - load, execute
  - get process attributes, set process attributes
  - wait event, signal event
  - allocate and free memory
- File management
  - create file, delete file
  - open, close
  - read, write, reposition
  - get file attributes, set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices
- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get process, file, or device attributes
  - set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach or detach remote devices
- Protection
  - get file permissions
  - set file permissions



We show here a table of how we will develop this in our CSOPESY emulator.

| System call             | Commercial OS | CSOPESY emulator                                                                                                                             |
|-------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Process control         | X             | Processes are simulated. Each process will have a pre-defined set of simple commands to execute → mock I/O read/write, print command, delay. |
| File management         | X             | File directory is simulated. All files are stored inside a single .csopesy file.                                                             |
| Device management       | X             | Simulated. Devices and contexts are configured.                                                                                              |
| Information maintenance | X             | Simulated, except no time-date support.                                                                                                      |
| Communications          | X             | Not supported.                                                                                                                               |
| Protections             | X             | No protection feature                                                                                                                        |

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



The figure above shows what a typical compiled C program executed in a commercial OS. How does it work in our emulator?

- C program → emulated process with list of commands
- System calls and its associated library is inverted → system calls are emulated, where standard C++ library calls are used to communicate with actual OS system calls.
- In other words, we have one “emulation” layer containing features that essentially make up a simple OS shell → process scheduler, memory management, file management, I/O and display interface. The emulation layer “simulates/translate/restricts” certain process commands as we create an OS that performs its duties. Recall operating system as coordinator.

**OTHER OPERATING SYSTEM SERVICES**

**Program execution**

- Basic idea: To run a program, it must be loaded into memory, and a register dictates the flow of instructions to be executed. How our programs going to be executed?
- As mentioned earlier, our representation of a program/process only consists of placeholder commands.
- The following will be devised: command flow (sequence of execution), process scheduling, resource and memory allocation.

**I/O operations**

- A running program may require I/O, which may involve a file or an I/O device. For specific devices, special functions may be desired (such as reading from a network interface or writing to a file system).
- For efficiency and protection, users usually cannot control I/O devices directly → e.g. directly controlling the printer head.
- In our OS emulator, request for a device/driver will be simulated → simply have a table of boolean/int flags, indicating the availability of arbitrary devices (e.g. available/unavailable, 3 devices, 1 device, etc).



File-system manipulation

- Programs would often need access to certain files and directories.
- They also need to create and delete themby name, search for a given file, and list file information.
- Some operating systems include permissions management to allow or deny access to files or directories based on file ownership.
- Emulator: Directory systems will be emulated → N editable files are “combined” together into a single .csopesy file.
- The .csopesy file resembles your disk storage. Multiple .csopesy files resembles multiple disk storages. E.g. C:/, D:/, E/

Communication

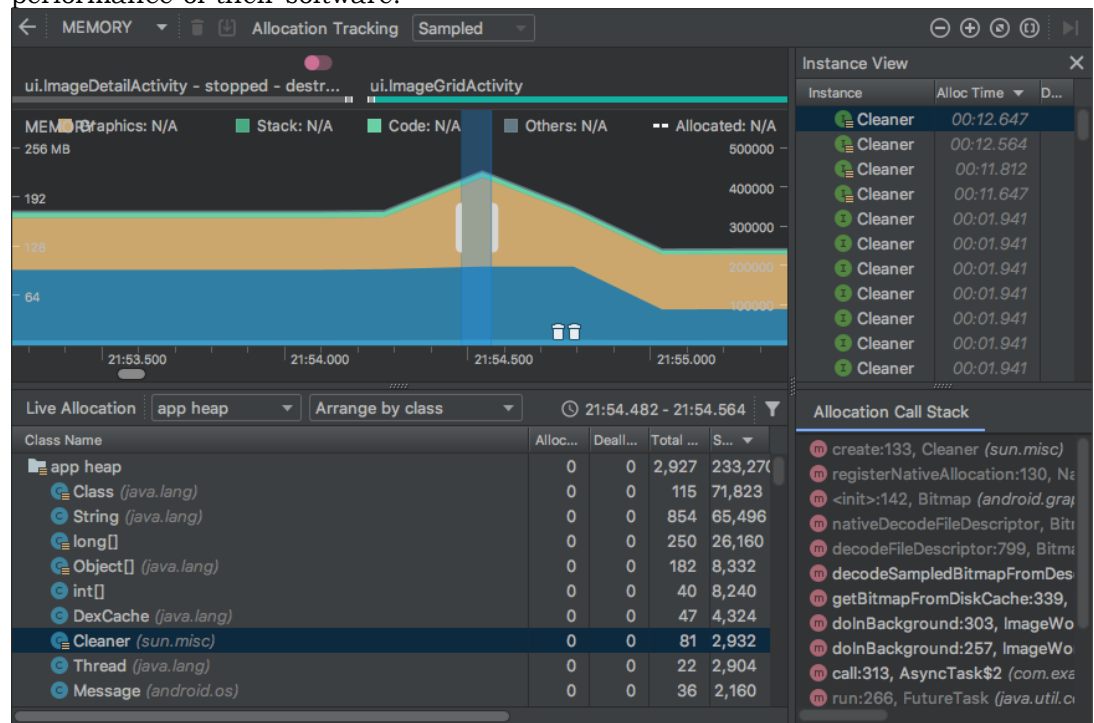
- Processes may exchange information with other processes. Scenario: Consider mobile OS (iOS/Android) → Active programs with running background services. Facebook application (foreground), sending notifications (background service).
- Some OS support concepts of shared memory across different processes. Example: Windows. Developers code their programs to access the shared memory pool: <https://learn.microsoft.com/en-us/windows/win32/memory/creating-named-shared-memory>
- Emulator: There will be no support for communication services.

Resource allocation

- When there are multiple processes running at the same time, resources must be allocated to each of them.
- The operating system manages many different types of resources. Some (such as CPU cycles, main memory, and file storage) may have special allocation codes, whereas others (such as I/O devices) may have much more general requests and release codes.
- For instance, in determining how best to use the CPU, operating systems have CPU-scheduling routines that take into account the speed of the CPU, the process that must be executed, the number of processing cores on the CPU, and other factors.
- There may also be routines to allocate printers, USB storage drives, and other peripheral devices.
- There must be a mechanism in dealing with possible deadlocks on resources → accessing a common resource across all processes.
- Emulator: The bulk of the emulator design would be on resource allocation.

Logging

- We want to keep track of which programs use how much and what kinds of computer resources.
- Most OS have a UI of usage statistics. Application developers also have means of profiling the performance of their software.



- Emulator: Basic logging must be supported. E.g. Viewing of running processes, checking resource availability, CPU and memory utilization, printing messages.

Protection and security

- Protection involves ensuring that all access to system resources is controlled. Some API calls must be restricted or granted privileges. E.g. Android application requesting for permission to access GPS.
- Security of the system would typically require each user to authenticate. Thus, OS must have user account support.
- Other security must be in place for programs and resources. E.g. Do not allow viewing of machine code.

- Emulator: We will not be implementing protection and security features.

### Activities

- Designing support for your **user interface - console states**. Recall that you created your placeholder commands. It's time to recognize these commands and create a new console layout for each command.
  - The drawing must only be done in the same console window.
- Hint on console state: There must be a mechanism to store different console layouts → refer to ConsoleManager for implementation.
- Using the reference console manager, provide proposed pseudocodes of its implementation. You may add additional functions. Be prepared to be called in class to present your proposed solution.

Some hints on designing console states. Header file of the console manager class is seen below.

```

1 #pragma once
2 #include <memory>
3 #include <vector>
4 #include "AConsole.h"
5 #include <unordered_map>
6 #include <Windows.h>
7 #include "TypedefRepo.h"
8
9 const String MAIN_CONSOLE = "MAIN_CONSOLE";
10 const String MARQUEE_CONSOLE = "MARQUEE_CONSOLE";
11 const String SCHEDULING_CONSOLE = "SCHEDULING_CONSOLE";
12 const String MEMORY_CONSOLE = "MEMORY_CONSOLE";
13
14 class ConsoleManager
15 {
16 public:
17 typedef std::unordered_map<String, std::shared_ptr<AConsole>> ConsoleTable;
18
19 static ConsoleManager* getInstance();
20 static void initialize();
21 static void destroy();
22
23 void drawConsole() const;
24 void process() const;
25 void switchConsole(String consoleName);
26 void returnToPreviousConsole();
27 void exitApplication();
28 bool isRunning() const;
29
30 HANDLE getConsoleHandle() const;
31
32 void setCursorPosition(int posX, int posY) const;
33
34 private:
35 ConsoleManager();
36 ~ConsoleManager() = default;
37 ConsoleManager(ConsoleManager const&) {} ; // copy constructor is private
38 ConsoleManager& operator=(ConsoleManager const&) {} ; // assignment operator is private*/
39 static ConsoleManager* sharedInstance;
40
41 ConsoleTable consoleTable;
42 std::shared_ptr<AConsole> currentConsole;
43 std::shared_ptr<AConsole> previousConsole;
44
45 HANDLE consoleHandle;
46 bool running = true;
47 };
48
49

```

The constructor of the console manager is seen below:

```

67 ConsoleManager::ConsoleManager()
68 {
69 this->running = true;
70
71 //initialize consoles
72 this->consoleHandle = GetStdHandle(STD_OUTPUT_HANDLE);
73
74 const std::shared_ptr<MainConsole> mainConsole = std::make_shared<MainConsole>();
75 const std::shared_ptr<MarqueeConsole> marqueeConsole = std::make_shared<MarqueeConsole>();
76 const std::shared_ptr<SchedulingConsole> schedulingConsole = std::make_shared<SchedulingConsole>();
77 const std::shared_ptr<MemorySimulationConsole> memoryConsole = std::make_shared<MemorySimulationConsole>();
78
79 this->consoleTable[MAIN_CONSOLE] = mainConsole;
80 this->consoleTable[MARQUEE_CONSOLE] = marqueeConsole;
81 this->consoleTable[SCHEDULING_CONSOLE] = schedulingConsole;
82 this->consoleTable[MEMORY_CONSOLE] = memoryConsole;
83
84 this->switchConsole(MAIN_CONSOLE);
85
86 }

```

**Goal:**

- Transition from console-based OS emulation to desktop-style GUI using ImGui (Dear ImGui library)
- Understand immediate-mode GUI paradigm and its advantages for real-time systems
- Implement polling mechanisms for keyboard and mouse input in a desktop environment
- Design responsive UI components that mirror modern OS interfaces

**Features to design in our CSOPESY emulator**

- A means to support keyboard input, both real-time and event-driven.
  - Real-time = The user can always type characters, and this displays on-screen immediately. E.g. Word processor programs.
    - This is also known as immediate-mode UI.
  - Event-driven = wait for an event, such as an enter command, and then perform an associated operation.
- How an application must behave under real-time/event-driven conditions.
  - Real-time = screen always refreshes even if there's no event/user input.
    - Sample applications: games and interactive applications like streaming websites.
    - Sample OSes: Modern OS like Windows are considered real-time.
  - Event-driven = screen refreshes after user input.
    - Sample applications:
    - Sample OSes: console/command line interfaces. However, there are means to support real-time switching. Example: Concept of tmux, pip download interface.

**Examples of real-time applications**

```
#include <iostream>
#include <conio.h>
#include <Windows.h> // For Sleep()

void keyboardInterruptHandler() {
 if (_kbhit()) {
 char key = _getch();
 std::cout << "Key pressed: " << key << std::endl;
 }
}

int main() {
 while (true) {
 // Simulate other program logic
 Sleep(100); // Sleep for 100 milliseconds

 // Check for keyboard input through the interrupt handler
 keyboardInterruptHandler();
 }

 return 0;
}
```

The example above shows a real-time keyboard polling mechanism.

Another example in CSOPESY emulator, where it shows the main loop. A real-time application would typically have a while loop wherein the application exits the loop when an exit/terminating sequence is triggered.

```

27 int main()
28 {
29 InputManager::initialize();
30 FileSystem::initialize();
31 // FileSystem::getInstance()->test_createRandomFiles(1000);
32 // FileSystem::getInstance()->saveFileSystem();
33 FileSystem::getInstance()->loadFileSystem();
34 ConsoleManager::initialize();
35 MessageBuffer::initialize();
36 ResourceEmulator::initialize();
37 MemoryManager::initialize();
38
39 bool running = true;
40 while(running)
41 {
42 ConsoleManager::getInstance()->process();
43 ConsoleManager::getInstance()->drawConsole();
44
45 running = ConsoleManager::getInstance()->isRunning();
46 }
47
48 // MemoryManager::test_MemoryAllocation();
49
50 MemoryManager::destroy();
51 ResourceEmulator::destroy();
52 MessageBuffer::destroy();
53 ConsoleManager::destroy();
54 InputManager::destroy();
55 return 0;
56 }

```

Another example in a game application where real-time interaction is highly valued:

```

// Game Loop
void RunGameLoop(IGameEvent& gameEvent) {
 const float targetFPS = 60.0f;
 const float frameTime = 1.0f / targetFPS;

 while (true) {
 // Handle user input
 if (_kbhit()) {
 char key = _getch();
 gameEvent.OnKeyPressed(key);
 }

 // Update game logic
 gameEvent.OnUpdate(frameTime);

 // Render game state
 gameEvent.OnRender();

 // Sleep to achieve a constant frame rate
 Sleep(static_cast<DWORD>(frameTime * 1000.0f));
 }
}

int main() {
 GameEventHandler gameHandler;

 // Start the game loop
 RunGameLoop(gameHandler);

 return 0;
}

```

### Examples of event-driven applications

Event-driven applications are those in which the flow of the program is determined by events such as user actions, sensor outputs, or messages from other programs

- Graphical User Interface (GUI) Applications
  - Example: A simple text editor
  - Events: Mouse clicks, keypresses, menu selections
  - Usage: When the user interacts with the GUI by clicking buttons, typing text, or choosing options from menus, events are triggered, and the application responds accordingly.

The code below shows Qt framework in C++, designed for event-driven applications:  
<https://www.qt.io/product/framework>

```

1 //The code below shows Qt framework in C++, designed for event-driven applications.
2 #include <QApplication>
3 #include <QTextEdit>
4 #include <QMenuBar>
5 #include <QMenu>
6 #include <QAction>
7 #include <QFileDialog>
8
9 class TextEditor : public QTextEdit {
10 Q_OBJECT
11
12 public:
13 TextEditor(QWidget *parent = nullptr) : QTextEdit(parent) {
14 createActions();
15 createMenus();
16 }
17
18 private:
19 void newFile() {
20 clear();
21 }
22
23 void openFile() {
24 //open file
25 }
26
27 void saveFile() {
28 //save file
29 }
30
31 private:
32 void createActions() {
33 newAction = new QAction(tr("&New"), this);
34 newAction->setShortcut(QKeySequence::New);
35 connect(newAction, &QAction::triggered, this, &TextEditor::newFile);
36
37 openAction = new QAction(tr("&Open"), this);
38 openAction->setShortcut(QKeySequence::Open);
39 connect(openAction, &QAction::triggered, this, &TextEditor::openFile);
40
41 saveAction = new QAction(tr("&Save"), this);
42 saveAction->setShortcut(QKeySequence::Save);
43 connect(saveAction, &QAction::triggered, this, &TextEditor::saveFile);
44 }
45
46 void createMenus() {
47 fileMenu = menuBar()->addMenu(tr("&File"));
48 fileMenu->addAction(newAction);
49 fileMenu->addAction(openAction);
50 fileMenu->addAction(saveAction);
51 }
52
53 QMenu *fileMenu;
54 QAction *newAction;
55 QAction *openAction;
56 QAction *saveAction;
57 };
58
59 int main(int argc, char *argv[]) {
60 QApplication app(argc, argv);
61
62 TextEditor textEditor;
63 textEditor.show();
64
65 return app.exec();
66 }

```



Web Applications

- Example: An online shopping website
- Events: Button clicks, form submissions, AJAX requests
- Usage: Users interact with the website by clicking buttons to add items to their cart, submitting forms for payment, and the application responds to these events.

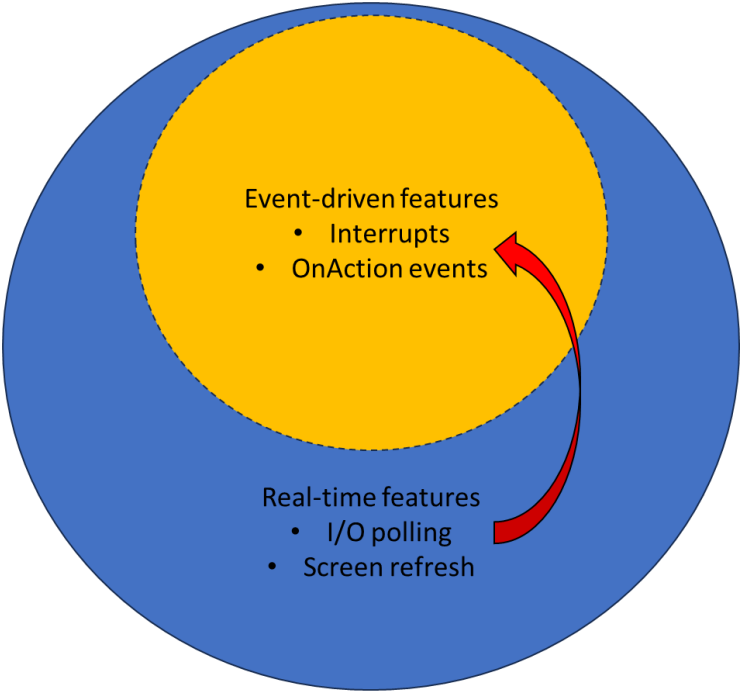
Mobile Applications:

- Example: A social media app
- Events: Taps, swipes, device orientation changes
- Usage: Users interact with the app by tapping on posts, swiping to navigate, and the application responds by displaying relevant content.

Networking Applications:

- Example: Chat application
- Events: Receiving a message, connection status changes
- Usage: When a user sends a message, or when the application receives a message from another user, events are triggered, and the application updates the chat interface.

**Observation:** All event-driven applications, are derived from real-time applications. All event-driven applications still require functionality that needs to run in real-time.



Family of implementation features

Keyboard polling example shown previously:  
Real-time feature → keyboard polling,  
Event-driven feature → IKeyboardEvent implementation

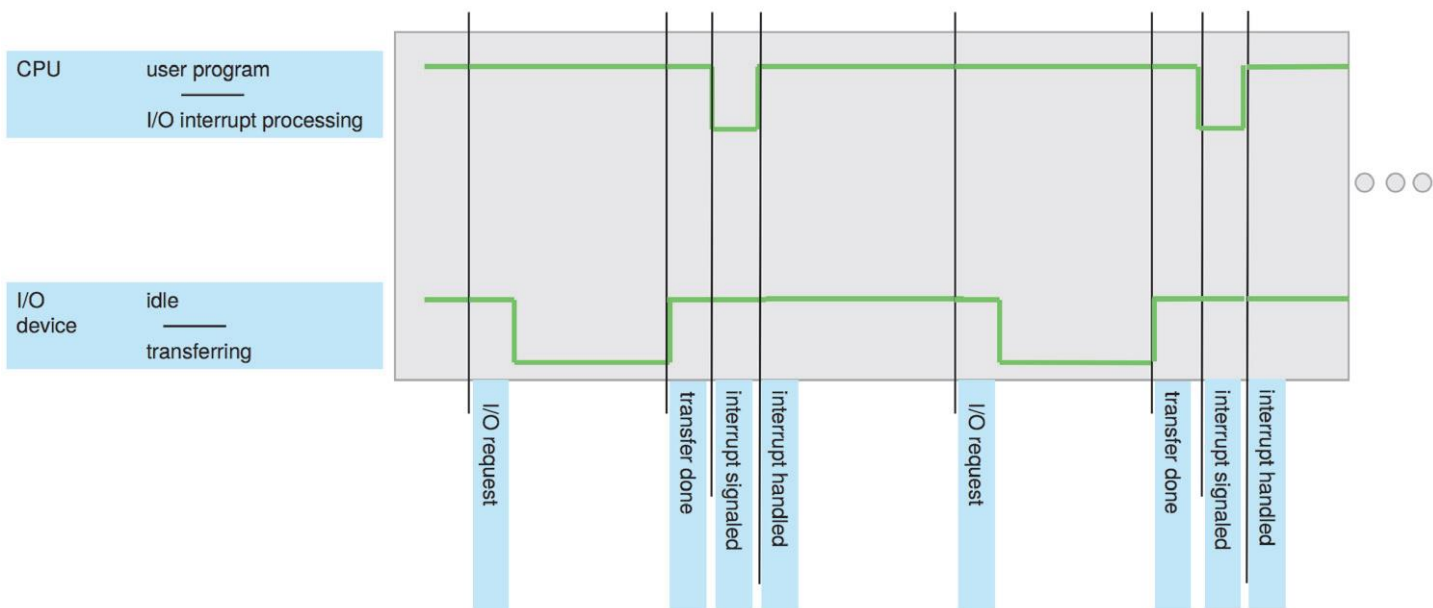
**Conceptual Framework: Polling vs. Interrupts**

To understand how real-time loops support event-driven functionality, we must define the two primary mechanisms for handling input/output (I/O) communication:

- **Polling (Active Checking):**
  - A mechanism where the application continuously and actively queries the status of an I/O device or buffer to see if an event has occurred.
  - In our console architecture, this is represented by the PollKeyboard function looping indefinitely and using \_kbhit() to check for key presses every cycle.
  - Pros: Extremely predictable timing; perfect for real-time applications (like games or emulators) that must refresh the screen every frame regardless of input.
  - Cons: Can result in high CPU utilization when idle due to busy-waiting.
- **Interrupts (Event-Driven Notification):**
  - A mechanism where the CPU or framework is alerted only when an external event occurs, temporarily suspending normal program execution to route control to an associated handler.
  - Instead of constantly asking "Is there data?", the system waits silently until it is interrupted by a trigger.
  - Pros: Highly efficient; dramatically reduces CPU wakeups when the application is idle.
  - Cons: Can introduce execution latency or frame-rate stutter in real-time graphic render pipelines if an interrupt handler stalls the loop.

- At the user application level, we mostly **deal with polling**. Interrupts are handled by event callbacks.
- Event callbacks act like a “queue” of interrupt events (e.g., very fast key presses happening before the next `_kbhit()` event).
- **Non-blocking vs blocking I/O**
  - Developers / APIs often have two ways to process I/O, which is either non-blocking or blocking.
  - `_kbhit()` is non-blocking. `getch()` is blocking.

Illustration of I/O interrupt processing, on a `_kbhit()` event.



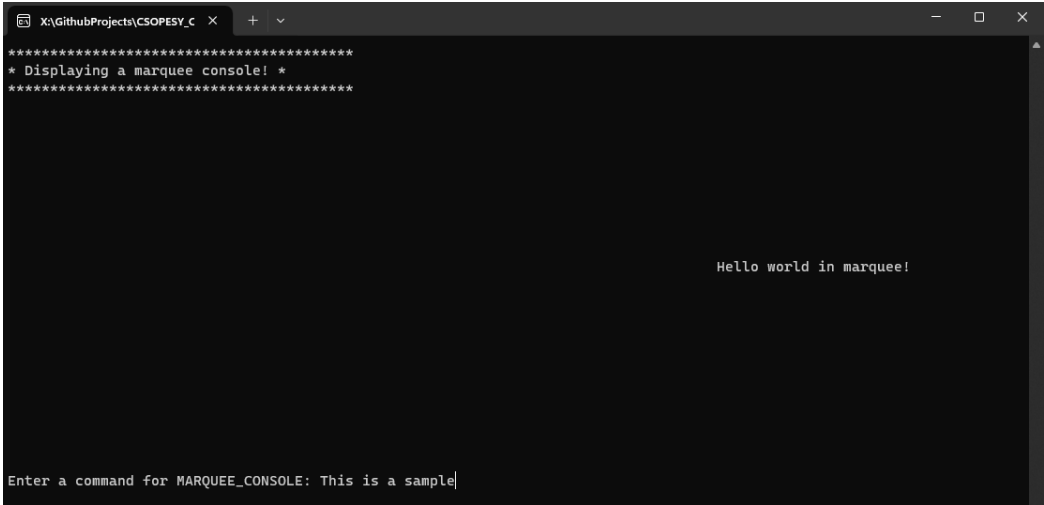
```
95 //Here is an example of a simple C++ application that combines real-time and event-driven handling:
96 #include <conio.h> // For _kbhit() and _getch()
97
98 // Interface for Keyboard Events
99 class IKeyboardEvent {
100 public:
101 virtual void OnKeyDown(char key) = 0;
102 virtual void OnKeyUp(char key) = 0;
103 };
104 // Implementation of IKeyboardEvent
105 class KeyboardEventHandler : public IKeyboardEvent {
106 public:
107 void OnKeyDown(char key) override {
108 std::cout << "Key Down: " << key << std::endl;
109 }
110
111 void OnKeyUp(char key) override {
112 std::cout << "Key Up: " << key << std::endl;
113 }
114 };
115 // Keyboard Polling
116 void PollKeyboard(IKeyboardEvent& keyboardEvent) {
117 while (true) {
118 if (_kbhit()) { // Check if a key has been pressed
119 char key = _getch(); // Get the pressed key
120
121 // Check if it's a key down or key up event
122 if (GetAsyncKeyState(key) & 0x8000) {
123 keyboardEvent.OnKeyDown(key);
124 } else {
125 keyboardEvent.OnKeyUp(key);
126 }
127 }
128
129 // Other program logic can continue here
130 }
131 }
132
133 int main() {
134 KeyboardEventHandler keyboardHandler;
135 // Start keyboard polling
136 PollKeyboard(keyboardHandler);
137 return 0;
138 }
```

- The `IKeyboardEvent` class defines an interface with `OnKeyDown` and `OnKeyUp` virtual functions.
- The `KeyboardEventHandler` class implements this interface, printing messages to the console when a key is pressed or released.
- The `PollKeyboard` function continuously checks for key presses using `_kbhit()` and retrieves the key using `_getch()`.
- Depending on the state of the key using `GetAsyncKeyState`, it calls the appropriate event handler.

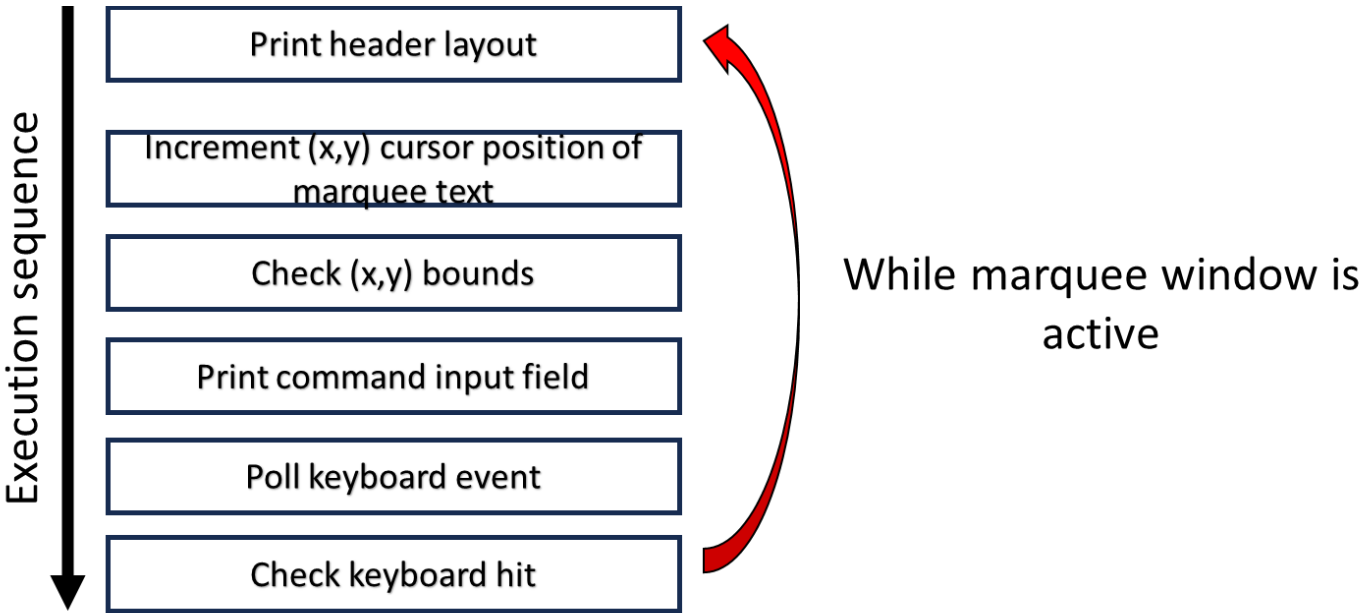
# IMPLEMENTING BASIC REAL-TIME APPLICATION USING KEYBOARD POLLING

- The goal is to support **real-time handling of I/O** while performing some CPU-based logic, such as a simple animation on the screen. This demonstrates the interleaving of I/O with CPU-based logic.
- Thus, for every cycle, we must have mechanisms to refresh the screen per frame, while polling for I/O events.

We shall demonstrate this with a marquee console → an animated text that moves across the screen per frame while still allowing the user to type in text.



General flow of the marquee console window is shown below:

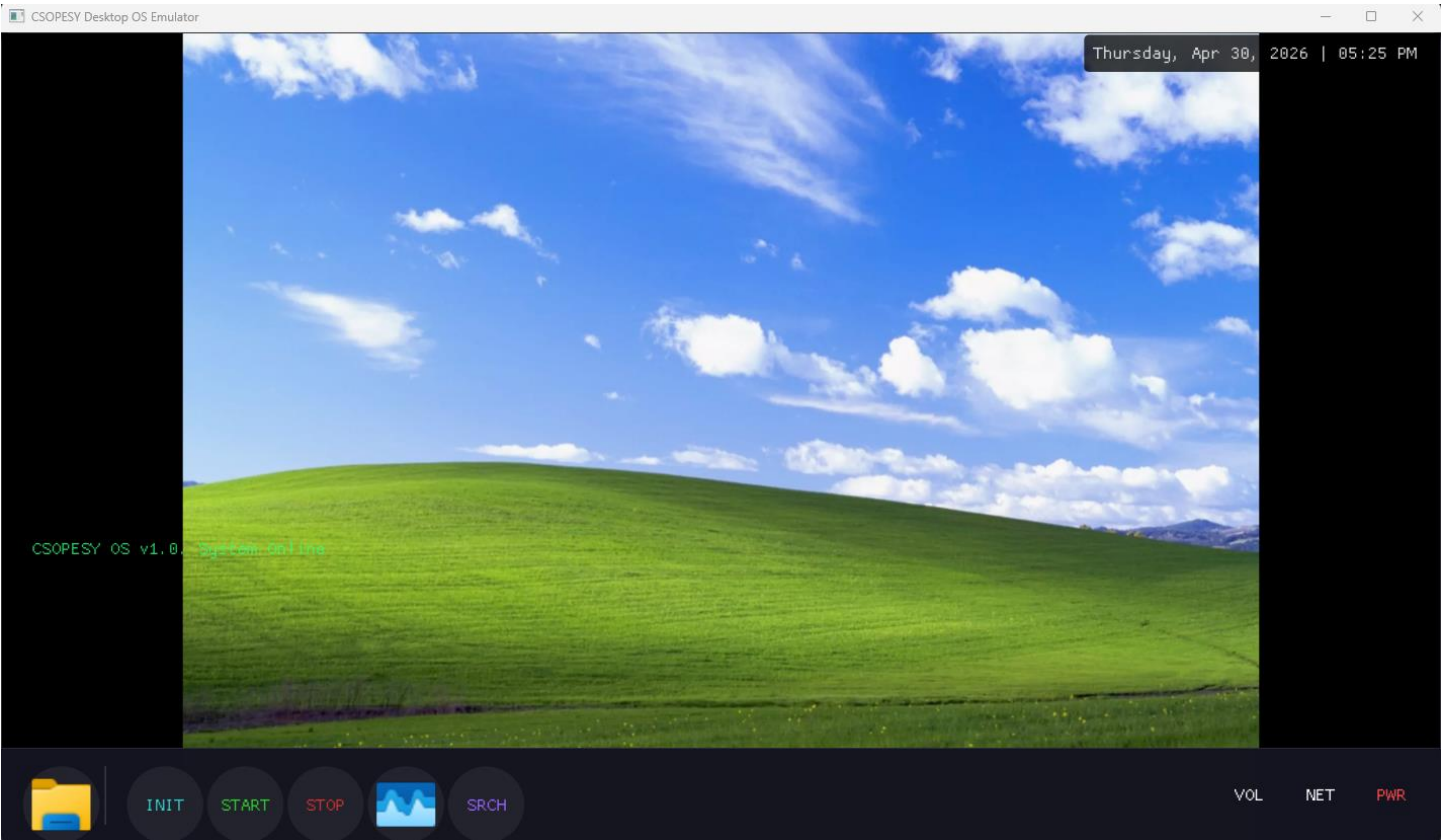
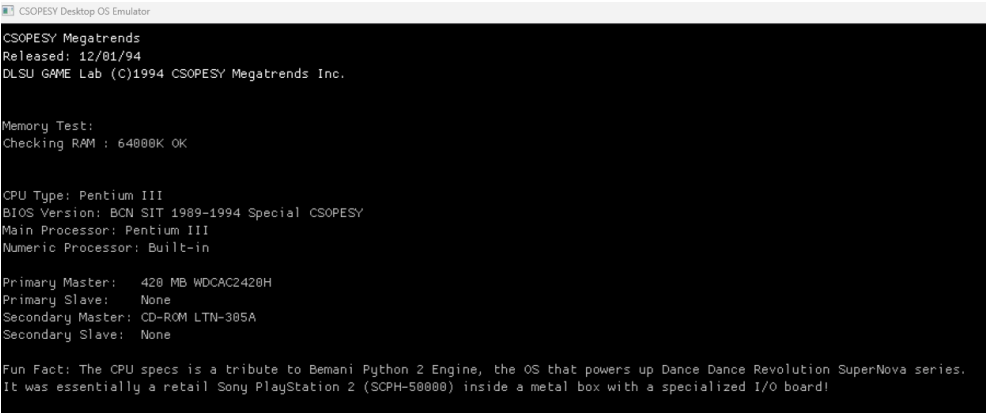


To create a customized console layout, we maximize the functionalities provided by the Windows Console API: <https://learn.microsoft.com/en-us/windows/console/console-functions>

```
114 void ConsoleManager::setCursorPosition(int posX, int posY) const
115 {
116 COORD coord;
117 coord.X = posX;
118 coord.Y = posY;
119 SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
120 }
```

# CREATING REAL-TIME INTERFACES / DESKTOP-STYLE OS

- An extension of the basic real-time application, by using an immediate-mode library available.
- Easiest/low-friction concept: Use an open-source immediate-mode library such as dear ImGui: <https://github.com/ocornut/imgui>





Why ImGui for OS?

- Simplicity: No complex widget trees or event hierarchies to manage
- Real-time focus: Perfect for systems that need per-frame updates (like OS emulators)
- Educational value: Students see the render loop explicitly, connecting to OS scheduling concepts
- Industry relevance: Used in game engines, real-time developer tools, and embedded systems.

Connection to Previous Weeks:

In Week 2, we covered the **five-phase OS loading sequence** (Bootstrapping → Kernel Init → System Services → Main Loop → Shutdown). Section 2.0 of this handout maps each of those phases to specific CSOPESY source files and functions.

In Weeks 4-5, we covered: • Console-based keyboard polling using `_kbhit()` and `_getch()` • Real-time vs event-driven application design • The `AConsole` class and `ConsoleManager` for multi-window console handling

In Weeks 6-7, we extend these concepts to: • Desktop GUI with windows, taskbars, and graphical widgets • GLFW-based event polling (`glfwPollEvents()`) • ImGui's immediate-mode rendering paradigm • Real-time data visualization (performance graphs, process tables)

2. From Console to Desktop: Evolution of the CSOPESY Emulator

2.0 Recap: A Typical Sequence of How an OS is Loaded and Run (Week 2 Review)

This subsection bridges Week 2's overview of the CSOPESY OS emulator with the desktop GUI concepts introduced in Weeks 6-7. Remind students that everything they are building — the boot window, the desktop, the taskbar — maps directly to the five-phase OS loading sequence they studied earlier. In Week 2, we established the **canonical five-phase sequence** by which any operating system — real or emulated — is loaded and begins running. This sequence is not just theoretical: every component you will build in Weeks 6-7 corresponds to a specific phase in this lifecycle.

The Five Phases mapped to UI sequence

| Phase                    | Description                                                   | CSOPESY Emulator Equivalent                                                             |
|--------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1. Bootstrapping         | Low-level hardware setup; load kernel into memory             | GUIApplication initializes GLFW + OpenGL; loads fonts and DPI config                    |
| 2. Kernel Initialization | Set up data structures, interrupt handlers, memory management | UIManager singleton created; UIConfig calculates DPI scale; window registry initialized |
| 3. Start System Services | Launch essential services, daemons, background processes      | Desktop, Taskbar, TaskManagerUI windows registered and made visible                     |



| Phase                   | Description                                                    | CSOPESY Emulator Equivalent                                                                      |
|-------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 4. Enter Main Loop      | Continuously handle interrupts, dispatch processes, handle I/O | GUIApplication::run() → glfwPollEvents() → ImGui::NewFrame() → renderFrame() → glfwSwapBuffers() |
| 5. Shutdown and Cleanup | Gracefully terminate processes, release memory, halt/reboot    | GLFW window close callback → ImGui::DestroyContext() → glfwTerminate()                           |

Phase 1 → Bootstrapping in CSOPESY

In a real OS, bootstrapping performs hardware initialization and loads the kernel. In CSOPESY, the equivalent is **GLFW and OpenGL initialization** inside GUIApplication:

```
// Phase 1: Bootstrapping – hardware/window system setup
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
GLFWwindow* window = glfwCreateWindow(width, height, "CSOPESY", nullptr, nullptr);
glfwMakeContextCurrent(window);

// Load OpenGL function pointers (equivalent to loading kernel into memory)
gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);

// Initialize Dear ImGui (equivalent to kernel entry point)
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");
```

**DEMO:** Open GUIApplication.cpp. Trace the constructor and initialize() method. Map each function call to the bootstrapping steps listed in Week 2 (hardware init → memory init → load kernel).

Phase 2 → Kernel Initialization in CSOPESY

Kernel initialization sets up data structures and memory management. In CSOPESY, this is **UIConfig and UIManager initialization** — the "kernel data structures" of the GUI subsystem:

```
// Phase 2: Kernel Initialization – set up data structures
UIConfig::getInstance()->calculateScaling(monitorWidth, monitorHeight);
UIManager::getInstance()->initialize();

// Set up "interrupt handlers" – GLFW callbacks
glfwSetKeyCallback(window, keyCallback);
glfwSetMouseButtonCallback(window, mouseButtonCallback);
glfwSetScrollCallback(window, scrollCallback);
```

**Observation:** UIManager is a **singleton registry** — analogous to the process table in a real kernel. Just as the OS kernel keeps a table of all running processes, UIManager keeps a registry of all active GUI windows.

Phase 3 → System Services in CSOPESY

The OS launches essential daemons and background services. In CSOPESY, this is **registering and showing the core GUI windows**:

```
// Phase 3: Start System Services – launch core windows
auto desktop = std::make_shared<Desktop>();
auto taskbar = std::make_shared<Taskbar>();
auto taskMgr = std::make_shared<TaskManagerUI>();
auto bootWindow = std::make_shared<BIOSBootWindow>();

UIManager::getInstance()->registerWindow("desktop", desktop);
UIManager::getInstance()->registerWindow("taskbar", taskbar);
UIManager::getInstance()->registerWindow("taskManager", taskMgr);
UIManager::getInstance()->registerWindow("boot", bootWindow);

UIManager::getInstance()->showWindow("boot"); // Start with boot sequence
```



**Observation** (from Week 2): "These are processes that are persistent in the OS — always alive / periodically invoking functions." The Desktop and Taskbar windows are exactly this: **persistent system services** that render every frame, just like OS daemons that are always running in the background.

**Phase 4 → Main Loop in CSOPESY**

The most critical phase: the OS enters an **infinite loop** to handle events, dispatch processes, and manage I/O. In CSOPESY, this is the **ImGui render loop**:

```
// Phase 4: Enter Main Loop - handle events, dispatch, I/O
while (!glfwWindowShouldClose(window)) {
 // Handle interrupts / poll for input events
 glfwPollEvents();

 // Dispatch "processes" - render all active windows
 ImGui::NewFrame();
 UIManager::getInstance()->renderAllWindows();
 ImGui::Render();

 // Flush output - present the rendered frame
 ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
 glfwSwapBuffers(window);
}
```

In the console-based emulator, the main loop used `_kbhit()` to poll for keyboard input. In the desktop GUI, `glfwPollEvents()` takes over this role — it polls **all** input devices (keyboard, mouse, window resize, etc.) and queues the events for ImGui to process. The polling *philosophy* is identical; only the mechanism changes.

**DEMO:** Place a breakpoint inside the `while` loop. Show students that this loop runs approximately 60 times per second (60 FPS). Ask: "What happens if one window's `draw()` function takes 100ms? How does that affect the entire system?"

**Phase 5 → Shutdown in CSOPESY**

When the user closes the window, the emulator must clean up — just as a real OS gracefully terminates processes and releases memory:

```
// Phase 5: Shutdown and Cleanup
ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext();
glfwDestroyWindow(window);
glfwTerminate();
```

**Observation:** Notice that cleanup is done in **reverse initialization order** — ImGui is shut down before GLFW, just as user-space processes are terminated before the kernel shuts down in a real OS.

**Mapping the BIOSBootWindow to Real BIOS/UEFI Boot**

The BIOSBootWindow is a deliberate pedagogical tool: it *simulates* the visual experience of a real BIOS/UEFI boot sequence. Compare the phases:

| Real BIOS/UEFI Phase      | BIOSBootWindow Phase | What the Student Sees                |
|---------------------------|----------------------|--------------------------------------|
| POST (Power-On Self Test) | BIOS_HEADER          | ASCII art logo, version info         |
| RAM check                 | RAM_CHECK            | Animated memory scan progress bar    |
| Hardware enumeration      | HARDWARE_INFO        | List of detected (simulated) devices |
| Boot settings display     | BIOS_SETTINGS        | Configuration summary                |
| OS handoff                | LOADING_OS           | "Loading CSOPESY OS..." spinner      |
| Kernel running            | COMPLETE             | Transition to Desktop                |

**Activity:** Ask students to open `BIOSBootWindow.cpp` and identify which `BootPhase` enum value corresponds to each of the five Week 2 OS loading phases. This is a quick 5-minute in-class mapping exercise.

# IMGUI Fundamentals: Immediate Mode vs Retained Mode

## What is Immediate Mode GUI?

**Definition:** An immediate-mode GUI rebuilds the entire interface **every frame** from scratch, with minimal retained state between frames [1].

### Contrast with Retained Mode:

| Aspect           | Retained Mode (e.g., Qt, WPF)               | Immediate Mode (e.g., Dear ImGui)        |
|------------------|---------------------------------------------|------------------------------------------|
| Widget storage   | Widgets stored in persistent tree/hierarchy | No persistent widget objects             |
| State management | Framework manages widget state              | Application manages state                |
| Update model     | Invalidate/repaint specific regions         | Rebuild entire UI each frame             |
| Memory overhead  | Higher (full widget tree)                   | Lower (minimal retained state)           |
| Code complexity  | Event handlers, callbacks, signals/slots    | Direct procedural code in render loop    |
| Best for         | Complex, persistent UIs (desktop apps)      | Real-time tools, debug overlays, editors |

## Why Immediate Mode for CSOPESY?

### Advantages for OS emulation:

1. **Simplicity:** Students write straightforward procedural code, not complex event hierarchies
2. **Real-time focus:** Natural fit for systems that update every frame (like OS schedulers)
3. **Dynamic layouts:** Easy to show/hide windows based on process state
4. **Educational clarity:** The render loop is explicit, connecting to OS scheduling concepts

## The IMGUI Philosophy

**Core principle:** "If you want to display a button, just call `Button()` every frame."

```
// Retained mode (Qt-style pseudocode)
Button* myButton = new Button("Click Me");
myButton->setOnClick([]{ doSomething(); });
layout->addWidget(myButton);

// Immediate mode (ImGui)
void renderFrame() {
 if (ImGui::Button("Click Me")) {
 doSomething();
 }
}
```

**Observation:** In immediate mode, the button doesn't "exist" between frames. The `Button()` function returns `true` if clicked **this frame**, and you handle it immediately.

# The IMGUI Render Loop: Architecture and Implementation

## The Main Application Loop

**Key file:** ``GUIApplication.cpp``

The heart of the IMGUI application is the **render loop**, which runs continuously until the user exits:

```
void GUIApplication::run() {
 while (!glfwWindowShouldClose(window)) {
 // 1. Poll input events (keyboard, mouse, window events)
 glfwPollEvents();

 // 2. Start new ImGui frame
 ImGui_ImplOpenGL3_NewFrame();
 ImGui_ImplGlfw_NewFrame();
 ImGui::NewFrame();

 // 3. Update application logic
 updateLogic();

 // 4. Render all UI components
 renderFrame();

 // 5. Finalize ImGui rendering
```

```

 ImGui::Render();

 // 6. Clear screen and render ImGui draw data
 int display_w, display_h;
 glfwGetFramebufferSize(window, &display_w, &display_h);
 glViewport(0, 0, display_w, display_h);
 glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
 glClear(GL_COLOR_BUFFER_BIT);
 ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

 // 7. Swap front and back buffers (double buffering)
 glfwSwapBuffers(window);
}
}

```

## Breaking Down the Render Loop

### Step 1: `glfwPollEvents()`

- Polls the OS for input events (keyboard, mouse, window resize, etc.)
- Non-blocking: returns immediately with all pending events
- GLFW translates OS-specific events into cross-platform callbacks
- ImGui's GLFW backend automatically captures these events via registered callbacks

**Connection to OS design:** This is analogous to the kernel's interrupt handler checking the interrupt queue. Modern OSes use **event-driven I/O** at the kernel level to reduce CPU wakeups [4].

### Step 2: `ImGui::NewFrame()`

- Prepares ImGui for a new frame of rendering
- Resets internal state (draw lists, input state, etc.)
- Captures input state from ImGuiIO (mouse position, key states, etc.)

### Step 3: `updateLogic()`

- Application-specific logic (process scheduling, memory management, etc.)
- Updates data structures that will be displayed in the UI
- In CSOPESY: updates process states, CPU utilization, memory usage

### Step 4: `renderFrame()`

- Calls all UI rendering functions (Desktop, Taskbar, Windows, etc.)
- Each component calls ImGui functions to declare its UI
- ImGui builds internal draw lists (vertices, indices, textures)

### Step 5: `ImGui::Render()`

- Finalizes ImGui's internal draw data
- Converts high-level widget calls into low-level rendering commands

### Step 6: OpenGL Rendering

- Clears the framebuffer
- Renders ImGui's draw data using OpenGL
- All UI is drawn to the back buffer

### Step 7: `glfwSwapBuffers()`

- Swaps front and back buffers (double buffering)
- Presents the rendered frame to the screen
- Typically synchronized with monitor refresh rate (VSync)

## Polling vs Event-Driven in GLFW

GLFW offers two input models:

1. **Polling** (`glfwPollEvents()`):
  - Checks for events once per frame
  - Non-blocking
  - Suitable for real-time applications (games, simulations)
  - Used in CSOPESY emulator
2. **Waiting** (`glfwWaitEvents()`):
  - Blocks until an event occurs
  - Reduces CPU usage when idle
  - Suitable for event-driven applications (text editors, IDEs)

**DEMO:** Modify `GUIApplication.cpp` to use `glfwWaitEvents()` instead of `glfwPollEvents()`. Show how the UI freezes until user input occurs. Discuss trade-offs.

# Building Desktop UI Components

## The Desktop: Full-Screen Background

Key file: `Desktop.cpp`

The Desktop is the foundation of the GUI, rendered as a full-screen ImGui window:

```
void Desktop::draw() {
 // Create full-screen window
 ImGui::SetNextWindowPos(ImVec2(0, 0));
 ImGui::SetNextWindowSize(ImGui::GetIO().DisplaySize);

 ImGui::Begin("Desktop", nullptr,
 ImGuiWindowFlags_NoTitleBar |
 ImGuiWindowFlags_NoResize |
 ImGuiWindowFlags_NoMove |
 ImGuiWindowFlags_NoCollapse |
 ImGuiWindowFlags_NoBringToFrontOnFocus);

 // Render wallpaper
 if (wallpaperTexture) {
 ImGui::GetWindowDrawList()->AddImage(
 (void*)(intptr_t)wallpaperTexture,
 ImVec2(0, 0),
 ImGui::GetIO().DisplaySize
);
 }

 // Render real-time clock
 drawClock();

 ImGui::End();
}

void Desktop::drawClock() {
 auto now = std::chrono::system_clock::now();
 auto time = std::chrono::system_clock::to_time_t(now);
 struct tm timeinfo;
 localtime_s(&timeinfo, &time);

 char buffer[64];
 strftime(buffer, sizeof(buffer), "%I:%M:%S %p", &timeinfo);

 ImVec2 textSize = ImGui::CalcTextSize(buffer);
 ImVec2 pos = ImVec2(ImGui::GetIO().DisplaySize.x - textSize.x - 20, 20);

 ImGui::SetCursorPos(pos);
 ImGui::TextColored(ImVec4(1, 1, 1, 1), "%s", buffer);
}
```

- Key techniques:**
- SetNextWindowPos/Size(): Position window before Begin()
  - Window flags: Remove title bar, disable resizing/moving
  - GetWindowDrawList(): Access low-level drawing API for custom rendering
  - CalcTextSize(): Measure text dimensions for positioning

**DEMO:** Show how removing `ImGuiWindowFlags_NoTitleBar` makes the desktop a movable window. Discuss why these flags are necessary.

## The Taskbar: Bottom Panel with Icons

Key file: `Taskbar.cpp`

The Taskbar is a horizontal panel at the bottom of the screen with clickable icons:

```
void Taskbar::draw() {
 ImVec2 displaySize = ImGui::GetIO().DisplaySize;
 float taskbarHeight = 60.0f * UIConfig::getScaleFactor();

 // Position at bottom of screen
 ImGui::SetNextWindowPos(ImVec2(0, displaySize.y - taskbarHeight));
 ImGui::SetNextWindowSize(ImVec2(displaySize.x, taskbarHeight));

 ImGui::Begin("Taskbar", nullptr,
 ImGuiWindowFlags_NoTitleBar |
 ImGuiWindowFlags_NoResize |
 ImGuiWindowFlags_NoMove |
 ImGuiWindowFlags_NoScrollbar);
}
```

```

// Render icons
ImGui::PushStyleVar(ImGuiStyleVar_ItemSpacing, ImVec2(10, 0));

for (auto& icon : taskbarIcons) {
 if (icon.texture) {
 if (ImGui::ImageButton((void*)(intptr_t)icon.texture,
 ImVec2(40, 40) * UIConfig::getScaleFactor())) {
 icon.onClick();
 }
 if (ImGui::IsItemHovered()) {
 ImGui::SetTooltip("%s", icon.name.c_str());
 }
 }
 ImGui::SameLine();
}

ImGui::PopStyleVar();

// System tray (right side)
drawSystemTray();

ImGui::End();
}

void Taskbar::drawSystemTray() {
 ImGui::SameLine(ImGui::GetWindowWidth() - 200);

 // CPU usage
 ImGui::Text("CPU: %.1f%%", getCPUUsage());
 ImGui::SameLine();

 // Memory usage
 ImGui::Text("MEM: %.1f%%", getMemoryUsage());
 ImGui::SameLine();

 // Clock
 ImGui::Text("%s", getCurrentTime().c_str());
}

```

#### Key techniques:

- `ImageButton()`: Clickable image widget
- `IsItemHovered()`: Detect mouse hover for tooltips
- `SameLine()`: Place widgets horizontally
- `PushStyleVar/PopStyleVar()`: Temporarily modify spacing
- `GetWindowWidth()`: Calculate right-aligned positions

**Observation:** The taskbar uses **DPI scaling** (`UIConfig::getScaleFactor()`) to ensure consistent size across different monitor resolutions. We'll cover this in detail in Section 9.

## Real-Time Data Visualization

### Task Manager: Performance Graphs

**Key file:** `TaskManagerUI.cpp`

The Task Manager displays real-time performance data using ImGui's plotting widgets:

```

void TaskManagerUI::draw() {
 if (!this->beginWindow()) return;

 // Tab bar for different views
 if (ImGui::BeginTabBar("TaskManagerTabs")) {
 if (ImGui::BeginTabItem("Performance")) {
 drawPerformanceTab();
 ImGui::EndTabItem();
 }
 if (ImGui::BeginTabItem("Processes")) {
 drawProcessesTab();
 ImGui::EndTabItem();
 }
 ImGui::EndTabBar();
 }

 this->endWindow();
}

```

```

void TaskManagerUI::drawPerformanceTab() {
 // CPU usage graph
 ImGui::Text("CPU Usage");
 ImGui::PlotLines("##CPU", cpuHistory.data(), cpuHistory.size(),
 0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

 // Memory usage graph
 ImGui::Text("Memory Usage");
 ImGui::PlotLines("##Memory", memoryHistory.data(), memoryHistory.size(),
 0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

 // Update history buffers
 updatePerformanceData();
}

void TaskManagerUI::updatePerformanceData() {
 // Shift history buffer
 for (int i = 0; i < cpuHistory.size() - 1; i++) {
 cpuHistory[i] = cpuHistory[i + 1];
 }

 // Add new data point
 cpuHistory[cpuHistory.size() - 1] = getCurrentCPUUsage();

 // Same for memory
 for (int i = 0; i < memoryHistory.size() - 1; i++) {
 memoryHistory[i] = memoryHistory[i + 1];
 }
 memoryHistory[memoryHistory.size() - 1] = getCurrentMemoryUsage();
}

```

### Key techniques:

- `PlotLines()`: Render line graph from array of floats
- `BeginTabBar/BeginTabItem()`: Create tabbed interface
- Circular buffer pattern: Shift old data, append new data

**Connection to real-time systems:** This is a classic **sliding window** visualization, common in system monitors (Windows Task Manager, macOS Activity Monitor, Linux htop).

### Process Table: Sortable Columns

```

void TaskManagerUI::drawProcessesTab() {
 if (ImGui::BeginTable("ProcessTable", 5,
 ImGuiTableFlags_Sortable |
 ImGuiTableFlags_Resizable |
 ImGuiTableFlags_Borders |
 ImGuiTableFlags_RowBg)) {

 // Setup columns
 ImGui::TableSetupColumn("PID", ImGuiTableColumnFlags_DefaultSort);
 ImGui::TableSetupColumn("Name");
 ImGui::TableSetupColumn("State");
 ImGui::TableSetupColumn("CPU %");
 ImGui::TableSetupColumn("Memory");
 ImGui::TableHeadersRow();

 // Get sort specs
 if (ImGuiTableSortSpecs* sortSpecs = ImGui::TableGetSortSpecs()) {
 if (sortSpecs->SpecsDirty) {
 sortProcesses(sortSpecs);
 sortSpecs->SpecsDirty = false;
 }
 }

 // Render rows
 for (const auto& process : processes) {
 ImGui::TableNextRow();

 ImGui::TableNextColumn();
 ImGui::Text("%d", process.pid);

 ImGui::TableNextColumn();
 ImGui::Text("%s", process.name.c_str());

 ImGui::TableNextColumn();
 ImVec4 stateColor = getStateColor(process.state);
 ImGui::TextColored(stateColor, "%s", getStateName(process.state));

 ImGui::TableNextColumn();
 ImGui::Text("%.1f%%", process.cpuUsage);

```

```

 ImGui::TableNextColumn();
 ImGui::Text("%zu KB", process.memoryUsage);
 }

 ImGui::EndTable();
}

ImVec4 TaskManagerUI::getStateColor(ProcessState state) {
 switch (state) {
 case ProcessState::RUNNING:
 return ImVec4(0.0f, 1.0f, 0.0f, 1.0f); // Green
 case ProcessState::READY:
 return ImVec4(1.0f, 1.0f, 0.0f, 1.0f); // Yellow
 case ProcessState::WAITING:
 return ImVec4(1.0f, 0.5f, 0.0f, 1.0f); // Orange
 case ProcessState::TERMINATED:
 return ImVec4(0.5f, 0.5f, 0.5f, 1.0f); // Gray
 default:
 return ImVec4(1.0f, 1.0f, 1.0f, 1.0f); // White
 }
}

```

#### Key techniques:

- `BeginTable()`: Create multi-column table with sorting/resizing
- `TableSetupColumn()`: Define column headers and properties
- `TableGetSortSpecs()`: Query user's sort preferences
- `TextColored()`: Color-code process states for visual clarity

**DEMO:** Show the Task Manager in action. Click column headers to sort. Discuss how this mirrors Windows Task Manager's design.

## 8. Window Management and State

### The AWindow Base Class

Key file: ``AWindow.cpp``

All windows in the CSOPESY emulator inherit from `AWindow`, which provides common functionality:

```

class AWindow {
public:
 AWindow(const std::string& name) : windowName(name), isVisible(false) {}
 virtual ~AWindow() = default;

 virtual void draw() = 0; // Pure virtual: subclasses must implement

 void show() { isVisible = true; }
 void hide() { isVisible = false; }
 bool isShown() const { return isVisible; }

protected:
 bool beginWindow() {
 if (!isVisible) return false;

 ImGui::Begin(windowName.c_str(), &isVisible);
 return true;
 }

 void endWindow() {
 ImGui::End();
 }

 std::string windowName;
 bool isVisible;
};

```

#### Key design patterns:

- **Template Method:** `draw()` is pure virtual; subclasses implement specific rendering
- **RAII-style:** `beginWindow()` / `endWindow()` pair ensures `ImGui::Begin()` / `ImGui::End()` are balanced
- **Visibility management:** `isVisible` flag controls whether window is rendered

**Observation:** The `&isVisible` parameter in `ImGui::Begin()` creates a close button (X) that automatically sets `isVisible = false` when clicked.



# The UIManager: Window Registry

Key file: `UIManager.cpp`

The UIManager maintains a registry of all windows and handles their lifecycle:

```
class UIManager {
public:
 static UIManager& getInstance() {
 static UIManager instance;
 return instance;
 }

 void registerWindow(const std::string& name, std::shared_ptr<AWindow> window) {
 windows[name] = window;
 }

 void showWindow(const std::string& name) {
 if (windows.find(name) != windows.end()) {
 windows[name]->show();
 }
 }

 void hideWindow(const std::string& name) {
 if (windows.find(name) != windows.end()) {
 windows[name]->hide();
 }
 }

 void updateAllWindows() {
 for (auto& [name, window] : windows) {
 if (window->isShown()) {
 window->update(); // Update logic (if needed)
 }
 }
 }

 void renderAllWindows() {
 for (auto& [name, window] : windows) {
 if (window->isShown()) {
 window->draw();
 }
 }
 }

private:
 std::map<std::string, std::shared_ptr<AWindow>> windows;
};
```

## Usage in main render loop:

```
void GUIApplication::renderFrame() {
 desktop->draw();
 taskbar->draw();
 UIManager::getInstance().renderAllWindows();
}
```

## Key design patterns:

- **Singleton:** Ensures single global registry
- **Registry pattern:** Central lookup for windows by name
- **Separation of concerns:** UIManager handles lifecycle, windows handle rendering

## Connection to OS design:

This mirrors how real OSes manage windows:

- Windows OS: Desktop Window Manager (DWM) maintains window list
- macOS: WindowServer manages window hierarchy
- Linux: X11/Wayland compositor tracks window state

---

## 9. DPI Scaling and Responsive Design

### The Problem: High-DPI Displays

Modern monitors have varying pixel densities:

- Standard HD: 1920×1080 (96 DPI)
- 4K: 3840×2160 (192 DPI)
- Retina: 2880×1800 (220 DPI)

**Without DPI scaling:** UI elements appear tiny on high-DPI displays.

**The Solution: UIConfig**

**Key file:** `UIConfig.cpp`

```
class UIConfig {
public:
 static void initialize() {
 // Get primary monitor
 GLFWmonitor* monitor = glfwGetPrimaryMonitor();
 const GLFWvidmode* mode = glfwGetVideoMode(monitor);

 // Calculate scale factor based on resolution
 float baseWidth = 1920.0f;
 float currentWidth = static_cast<float>(mode->width);
 scaleFactor = currentWidth / baseWidth;

 // Clamp to reasonable range
 if (scaleFactor < 1.0f) scaleFactor = 1.0f;
 if (scaleFactor > 2.0f) scaleFactor = 2.0f;
 }

 static float getScaleFactor() {
 return scaleFactor;
 }

 static ImVec2 scale(ImVec2 size) {
 return ImVec2(size.x * scaleFactor, size.y * scaleFactor);
 }

private:
 static float scaleFactor;
};

float UIConfig::scaleFactor = 1.0f;
```

**Usage in UI components:**

```
// Taskbar.cpp
float taskbarHeight = 60.0f * UIConfig::getScaleFactor();

// ProcessWindow.cpp
ImVec2 windowSize = UIConfig::scale(ImVec2(600, 400));
ImGui::SetNextWindowSize(windowSize, ImGuiCond_FirstUseEver);
```

**ImGui's built-in DPI support:**

```
void GUIApplication::setupImGui() {
 ImGuiIO& io = ImGui::GetIO();

 // Enable DPI awareness
 io.ConfigFlags |= ImGuiConfigFlags_DpiEnableScaleFonts;
 io.ConfigFlags |= ImGuiConfigFlags_DpiEnableScaleViewports;

 // Load font with appropriate size
 float fontSize = 16.0f * UIConfig::getScaleFactor();
 io.Fonts->AddFontFromFileTTF("fonts/Roboto-Regular.ttf", fontSize);
}
```

**DEMO:** Run the emulator on different monitor resolutions (or simulate with window resizing). Show how UI elements scale proportionally.

**Practice Exercises and Activities**

**Exercise 1: Custom Desktop Widget**

**Objective:** Create a custom desktop widget that displays real-time system information.

**Requirements:**

- 1. Inherit from AWindow
- 2. Display at least 3 real-time metrics (e.g., CPU usage, memory usage, network activity)
- 3. Update every frame
- 4. Position in a fixed location on the desktop (e.g., top-right corner)
- 5. Use color-coding to indicate status (green = normal, yellow = warning, red = critical)

**Starter code:**

```

class SystemMonitorWidget : public AWindow {
public:
 SystemMonitorWidget() : AWindow("System Monitor") {}

 void draw() override {
 if (!this->beginWindow()) return;

 // TODO: Display CPU usage
 // TODO: Display memory usage
 // TODO: Display network activity
 // TODO: Color-code based on thresholds

 this->endWindow();
 }
};

```

#### Discussion questions:

- How often should you update the metrics? Every frame, or less frequently?
- What are the performance implications of querying system metrics every frame?
- How would you implement a "minimize to taskbar" feature?

## Exercise 2: Animated Boot Sequence

**Objective:** Extend BIOSBootWindow with additional boot phases and animations.

#### Requirements:

1. Add at least 2 new boot phases (e.g., "Loading Drivers", "Starting Services")
2. Each phase should have a progress bar that fills over time
3. Display a list of items being loaded (e.g., "Loading driver: USB Controller")
4. Use ImGui::GetTime() for timing
5. Transition to desktop when complete

#### Starter code:

```

enum class BootPhase {
 BIOS_LOGO,
 MEMORY_CHECK,
 DEVICE_INIT,
 LOADING_DRIVERS, // New phase
 STARTING_SERVICES, // New phase
 BOOT_COMPLETE
};

void BIOSBootWindow::drawLoadingDrivers() {
 ImGui::Text("Loading Drivers...");

 // TODO: Display list of drivers
 // TODO: Show progress bar
 // TODO: Advance to next phase when complete
}

```

#### Discussion questions:

- How does this boot sequence compare to a real OS boot (e.g., Windows, Linux)?
- What information would a real BIOS/UEFI display during boot?
- How would you implement a "skip boot animation" feature?

## Exercise 3: Process Window Enhancements

**Objective:** Add interactive features to ProcessWindow.

#### Requirements:

1. Add a "Pause/Resume" button that changes process state
2. Add a "Kill Process" button with confirmation dialog
3. Display a real-time progress bar for process completion
4. Add a "Priority" dropdown to change process scheduling priority
5. Log all user actions to the command log

#### Starter code:

```

void ProcessWindow::draw() {
 if (!this->beginWindow()) return;

```

```

// Existing code...

// TODO: Add Pause/Resume button
if (ImGui::Button("Pause")) {
 // ...
}
ImGui::SameLine();
if (ImGui::Button("Kill")) {
 // TODO: Show confirmation dialog
}

// TODO: Add priority dropdown
const char* priorities[] = { "Low", "Normal", "High", "Real-time" };
static int currentPriority = 1;
if (ImGui::Combo("Priority", ¤tPriority, priorities, 4)) {
 // TODO: Update process priority
}

this->endWindow();
}

```

#### Discussion questions:

- How do real OSes implement process priority? (Hint: scheduling algorithms)
- What are the security implications of allowing users to kill any process?
- How would you implement a "Force Kill" feature for unresponsive processes?

## Exercise 4: Task Manager Performance Graphs

**Objective:** Enhance TaskManagerUI with additional performance visualizations.

#### Requirements:

1. Add a "Disk Usage" graph
2. Add a "Network Usage" graph
3. Display min/max/average values for each metric
4. Allow user to change graph time range (e.g., 30s, 1m, 5m)
5. Add a "Pause" button to freeze the graphs

#### Starter code:

```

void TaskManagerUI::drawPerformanceTab() {
 // Existing CPU and memory graphs...

 // TODO: Add disk usage graph
 ImGui::Text("Disk Usage");
 ImGui::PlotLines("##Disk", diskHistory.data(), diskHistory.size(),
 0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

 // TODO: Display statistics
 ImGui::Text("Min: %.1f%% | Max: %.1f%% | Avg: %.1f%%",
 getMin(diskHistory), getMax(diskHistory), getAvg(diskHistory));

 // TODO: Add time range selector
 const char* ranges[] = { "30s", "1m", "5m" };
 static int currentRange = 0;
 if (ImGui::Combo("Time Range", ¤tRange, ranges, 3)) {
 resizeHistoryBuffers(currentRange);
 }
}

```

#### Discussion questions:

- How does Windows Task Manager implement its performance graphs?
- What are the trade-offs between graph resolution and memory usage?
- How would you implement a "Export to CSV" feature?

## Exercise 5: Multi-Desktop Support

**Objective:** Implement virtual desktops (like Windows 10+ Task View).

#### Requirements:

1. Support at least 3 virtual desktops
2. Each desktop has its own set of open windows
3. Add a desktop switcher UI (e.g., thumbnails in taskbar)

- 4. Animate transitions between desktops (e.g., slide effect)
- 5. Allow dragging windows between desktops

**Starter code:**

```
class DesktopManager {
public:
 void addDesktop() {
 desktops.push_back(std::make_shared<Desktop>());
 }

 void switchToDesktop(int index) {
 if (index >= 0 && index < desktops.size()) {
 currentDesktopIndex = index;
 // TODO: Animate transition
 }
 }

 void renderCurrentDesktop() {
 if (currentDesktopIndex < desktops.size()) {
 desktops[currentDesktopIndex]->draw();
 }
 }

private:
 std::vector<std::shared_ptr<Desktop>> desktops;
 int currentDesktopIndex = 0;
};
```

**Discussion questions:**

- How do real OSes implement virtual desktops? (Hint: window visibility flags)
- What are the memory implications of multiple desktops?
- How would you implement a "Show all desktops" overview (like macOS Mission Control)?

---

**Activity: Group Presentation**

**Objective:** Each student group implements one of the exercises above and presents to the class.

**Format:**

1. **Demo** (5 minutes): Show your implementation running
2. **Code walkthrough** (5 minutes): Explain key design decisions
3. **Challenges** (3 minutes): Discuss difficulties encountered and how you solved them
4. **Q&A** (2 minutes): Answer questions from classmates and instructor

**Evaluation criteria:**

- **Functionality:** Does it meet all requirements?
- **Code quality:** Is the code well-structured and readable?
- **UI/UX:** Is the interface intuitive and responsive?
- **Presentation:** Is the explanation clear and engaging?

---

**Activity: Performance Profiling**

**Objective:** Measure and optimize the CSOPESY emulator's performance.

**Tasks:**

1. **Measure frame rate:** Use `ImGui::GetIO().Framerate` to display FPS
2. **Identify bottlenecks:** Use a profiler (e.g., Visual Studio Profiler, Valgrind) to find slow functions
3. **Optimize:** Reduce unnecessary computations in the render loop
4. **Compare:** Measure FPS before and after optimization

**Guiding questions:**

- What is the target frame rate for a desktop application? (Hint: 60 FPS)
- What operations should be done every frame vs. less frequently?
- How does the number of open windows affect performance?

**Report format:**

- **Baseline:** FPS with 0, 5, 10, 20 windows open
- **Bottlenecks:** Top 3 slowest functions (with profiler screenshots)
- **Optimizations:** What you changed and why
- **Results:** FPS improvement after optimization

---

## 12. Discussion Questions

### Conceptual Questions

1. **Immediate vs Retained Mode:**
  - Why is immediate-mode GUI simpler to implement than retained-mode?
  - In what scenarios would retained-mode be preferable?
  - How does ImGui's paradigm relate to functional programming concepts?
2. **Polling vs Event-Driven:**
  - Why do real-time applications (games, simulations) use polling?
  - Why do event-driven applications (text editors, IDEs) wait for events?
  - Can you design a hybrid approach that combines both?
3. **OS Design:**
  - How does the CSOPESY emulator's window management compare to Windows' DWM?
  - What are the advantages of a compositing window manager?
  - How would you implement window transparency and blur effects?
4. **Performance:**
  - What is the performance cost of rebuilding the UI every frame?
  - How can you minimize draw calls in ImGui?
  - What is the role of GPU acceleration in modern UI rendering?

### Implementation Questions

5. **DPI Scaling:**
  - Why is DPI scaling necessary for modern applications?
  - How does the CSOPESY emulator's UIConfig approach compare to OS-level scaling?
  - What are the challenges of supporting multiple monitors with different DPIs?
6. **9. State Management:**
  - How does ImGui manage widget state (e.g., text input buffers)?
  - What is the role of widget IDs in ImGui?
  - How would you implement persistent window positions across sessions?
7. **10. Rendering Pipeline:**
  - What is the purpose of double buffering (`glfwSwapBuffers()`)?
  - How does VSync affect frame rate and input latency?
  - What is the difference between immediate-mode rendering and deferred rendering?

Overview

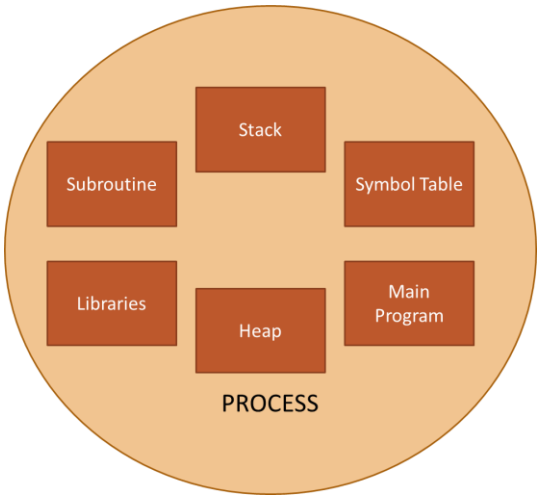
- How processes must be represented in code.
- Process scheduling is the heart of any operating system.
- Almost all platforms would have multiple processes running at the same time, requesting certain resources.
- The goal of the OS is to ensure that all processes execute, keeping the compute units (CPU and GPU) busy, while keeping the overall hardware stable.

Before we dive deep into CPU scheduling, we need to formalize the definition of a **process**, as these are the ones that will use the CPU.

INTRODUCTION TO PROCESSES

A process is a "**program in execution.**" It is more than just the executable code; it includes current activity, represented by the value of the program counter and the contents of the processor's registers.

- One program = one process.
- A standalone application that contains memory, stack, heap, etc.
- In C++, we use fork() method to create a duplicate process of the program.
- Context switch between the process is time consuming.
- We don't typically practice developing programs using multiple processes.
- Since we are simply going to emulate processes in the future, we represent processes as threads in the CSOPESY emulator.



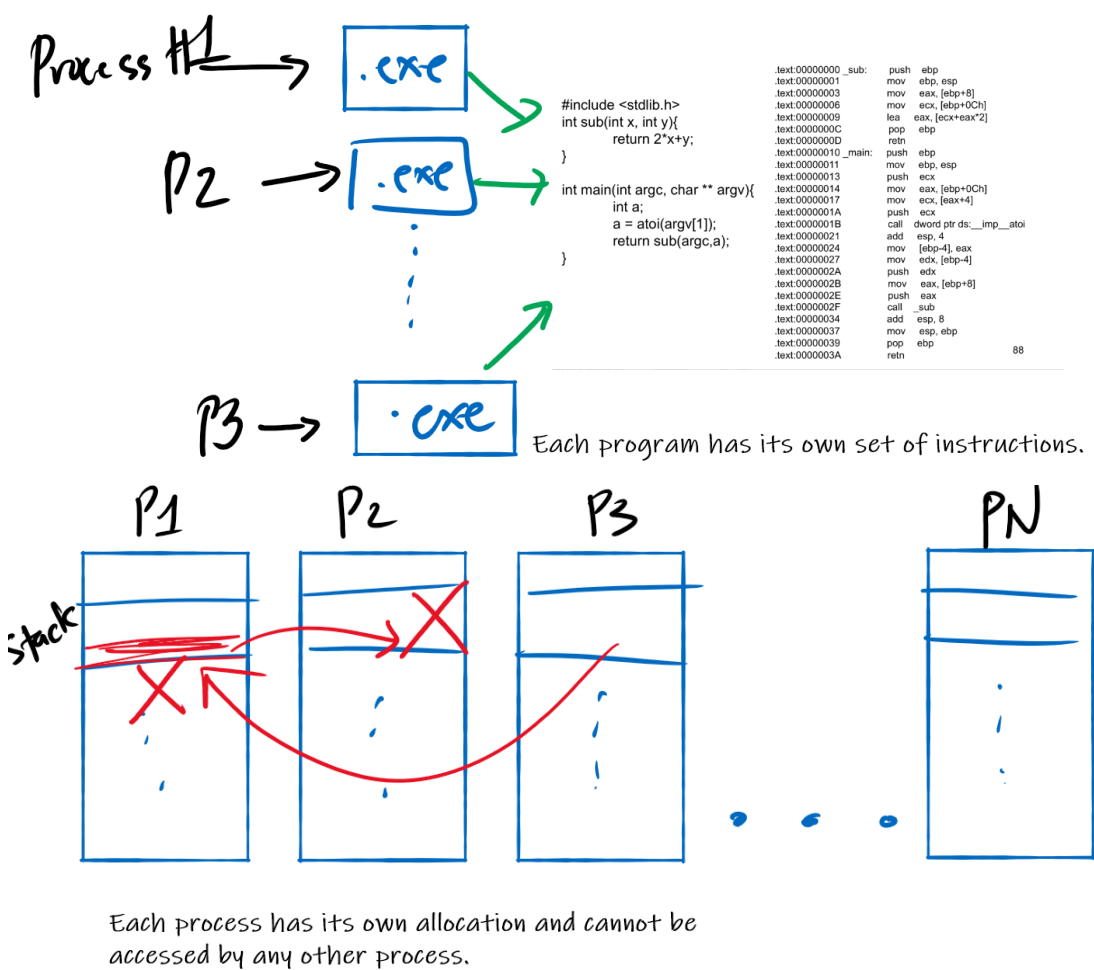
Components of a Program

- Main Program – points to the main function
- Subroutine – contains all functions declared in the code
- Libraries – external functions or dependencies required
- Symbol table – holds variables
- Stack – holds temporary variables for function calls. Also stores the return address of a given function call.
- Heap – memory for dynamically allocated objects.

| Program components | Real-world | Remarks                                                   | Machine project/OS emulator | Remarks                                                                                        |
|--------------------|------------|-----------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------|
| Main program       | ✓          |                                                           | ✓                           |                                                                                                |
| Subroutine         | ✓          | User-defined functions (e.g. creating factorial function) | ✓                           | User-defined instructions like ADD, READ, WRITE are analogous to subroutines in real-world OS. |
| Libraries          | ✓          | Programs often link to external libraries for             | ✗                           | The OS emulator does not explicitly support external libraries or                              |



|              |   |                                                                                           |   |                                                                                                                                        |
|--------------|---|-------------------------------------------------------------------------------------------|---|----------------------------------------------------------------------------------------------------------------------------------------|
|              |   | extended functionality and dependencies                                                   |   | dependencies for processes.                                                                                                            |
| Symbol table | ✓ | The symbol table holds variables and other symbols, mapping them to memory locations.     | ✓ | The OS emulator explicitly states that "uint16 variables are stored in the symbol table segment of the process".                       |
| Stack        | ✓ | The stack is used for temporary variables, function call arguments, and return addresses. | X |                                                                                                                                        |
| Heap         | ✓ | The heap is used for dynamic memory allocation during program execution.                  | ✓ | Read and write instructions operate on memory addresses where values remain persistent.<br><br>There must be a dedicated memory space. |



In the operating system, each process is represented by a **Process Control Block (PCB)**. A typical PCB contains:

- Process State
- Program Counter
- CPU Registers
- CPU-Scheduling Information
- Memory-Management Information

- Accounting Information
- I/O Status Information

## C++ Representation of a Process

In our emulator, we represent the PCB using a `Process` class. This class manages the process lifecycle, its instructions, and its local variables.

```
Proposed `Process.h`
```cpp
#pragma once
#include <memory>
#include <vector>
#include <string>
#include "ICommand.h"
#include "SymbolTable.h"

class Process {
public:
    enum ProcessState {
        READY,
        RUNNING,
        WAITING,
        FINISHED
    };

    Process(int pid, std::string name);

    void addCommand(std::shared_ptr<ICommand> command);
    void executeCurrentCommand();
    void moveToNextLine();

    bool isFinished() const;
    int getPID() const;
    ProcessState getState() const;
    std::string getName() const;

    // Symbol Table access
    SymbolTable& getSymbolTable();

private:
    int pid;
    std::string name;
    ProcessState currentState;
    int commandCounter;

    std::vector<std::shared_ptr<ICommand>> commandList;
    SymbolTable symbolTable;
};
```
```

## xv6 struct proc Expansion

In xv6, the struct proc serves as the Process Control Block. Below is the definition from the xv6-riscv version:

```
// Per-process state
struct proc {
 struct spinlock lock;

 // p->lock must be held when using these:
```

```

enum procstate state; // Process state
void *chan; // If non-zero, sleeping on chan
int killed; // If non-zero, have been killed
int xstate; // Exit status to be returned to parent's wait
int pid; // Process ID

// wait_lock must be held when using this:
struct proc *parent; // Parent process

// these are private to the process, so p->lock need not be held.
uint64 kstack; // Virtual address of kernel stack
uint64 sz; // size of process memory (bytes)
pagetable_t pagetable; // Pythecal page table
struct trapframe *trapframe; // data page for trampoline.S
struct context context; // swtch() here to run process
struct file *ofile[NOFILE]; // Open files
struct inode *cwd; // Current directory
char name[16]; // Process name (debugging)
};

```

### Why is spinlock needed in struct proc?

In a multiprocessor system like xv6, multiple CPUs may attempt to access or modify the same process simultaneously. The spinlock protects the integrity of the process's state and other fields.

- **State Atomicity:** Prevents race conditions where two CPUs might try to schedule or wake up the same process at the same time.
- **Critical Sections:** Functions like `yield()`, `sleep()`, and `wakeup()` acquire the `p->lock` to ensure that state transitions (e.g., from `RUNNING` to `SLEEPING`) happen atomically without being interrupted or observed in an inconsistent state by the scheduler.

### Key Fields:

- **state:** Defines the current lifecycle of the process. In xv6, these are defined in `enum procstate`:
- `enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };`
- **sz:** Tracks the size of the process memory in bytes, which is critical for memory management and allocation.
- **pid:** A unique identifier for the process, used by the kernel to manage and schedule it.
- **context:** Contains the register state (like `ra`, `sp`, `s0-s11`) used by the scheduler to perform context switching.
- **pagetable:** Points to the process's private page table, ensuring memory isolation between processes.

---

## Representing process instructions

The general idea is to create a command/instruction interface that all other instructions will implement. E.g., `print`, `add`, `read`, `write`, etc.

Below is a simple example of how to implement a `print` command, without variable assignments:

```

5 class ICommand
6 {
7 public:
8 enum CommandType
9 {
10 IO,
11 PRINT
12 };
13
14 ICommand(int pid, CommandType commandType);
15 CommandType getCommandType();
16 virtual void execute();
17
18 protected:
19 int pid;
20 CommandType commandType;
21 };
22
23 inline ICommand::CommandType ICommand::getCommandType()
24 {
25 return this->commandType;
26 }
27
28 inline void ICommand::execute()
29 {
30 IETThread::sleep(ms::Delay::PER_INSTRUCTION_DELAY);
31 }
32
33 inline ICommand::ICommand(int pid, CommandType commandType)
34 {
35 this->pid = pid;
36 this->commandType = commandType;
37 }

```

```

4 class PrintCommand : public ICommand
5 {
6 public:
7 PrintCommand(int pid, String& toPrint);
8 void execute() override;
9 private:
10 String toPrint;
11 };

```

```

5 PrintCommand::PrintCommand(int pid, String& toPrint) : ICommand(pid, CommandType::PRINT)
6 {
7 this->toPrint = toPrint;
8 }
9
10 void PrintCommand::execute()
11 {
12 ICommand::execute();
13
14 // std::cout << "PID " << this->pid << ":" << this->toPrint << std::endl;
15 std::stringstream msg; msg << String(Ptr: "PID ") << this->pid << ":" << this->toPrint << std::endl;
16 MessageBuffer::log(message: msg.str());
17 }

```

Rudimentary example of a variable declaration command

```

712 class DeclareCommand : public ICommand
713 {
714 public:
715 // Constructor for DECLARE: takes variable name and initial value
716 DeclareCommand(int pid, String& varName, uint16_t value) : ICommand(pid, CommandType::DECLARE)
717 {
718 this->varName = varName;
719 this->value = value;
720 //IMPORTANT: Variables are only stored temporarily. To make them "appear" or be associated in the process,
721 //the actual declaration must occur in the execute() command.
722 }
723
724 void execute() override;
725
726 //TODO: The actual declaration --> symbol table association + placement in memory.
727 //NOTE: Consider using std::move or move-based operation for transferring the data without copying.
728 void performDeclaration();
729
730 private:
731 String varName;
732 uint16_t value; // uint16_t for the value
733 };
734
735 void DeclareCommand::execute()
736 {
737 ICommand::execute(); // Call base class execute() for common behavior
738
739 performDeclaration(); //perform the actual declaration and storing in symbol table + memory.
740
741 std::stringstream msg;
742 msg << "Executing DECLARE command for PID " << this->pid
743 | << ": variable '" << this->varName << "' with value " << this->value << std::endl;
744 std::cout << msg.str();
745 }

```

### Activities:

- Redesign the print command such that it can accept and print one variable as stated in the specs.
- Design the variable declaration command.
- Design the remaining commands: ADD, SUBTRACT, SLEEP, FOR. For each command, show the header implementation and provide a documentation/report discussing your planned implementation.

# Symbol Table

A process needs to store and retrieve variables during execution. The **Symbol Table** maps variable names (strings) to their values (e.g., integers).

## Proposed SymbolTable.h

```
#pragma once
#include <string>
#include <unordered_map>
#include <cstdint>

class SymbolTable {
public:
 void setVariable(const std::string& name, int value) {
 table[name] = value;
 }

 int getVariable(const std::string& name) {
 if (table.find(name) != table.end()) {
 return table[name];
 }
 return 0; // Default or error value
 }

 bool hasVariable(const std::string& name) const {
 return table.find(name) != table.end();
 }

private:
 std::unordered_map<std::string, int> table;
};
```

---

### Note: Where is the Symbol Table in xv6?

In the C++ Emulator, we use an explicit SymbolTable class (or std::unordered\_map) stored in the PCB to track variables. In **xv6 (and real operating systems)**, the "Symbol Table" does not exist inside the PCB at runtime.

- **During Compilation:** The compiler/linker creates a symbol table and stores it in the **ELF executable file** (e.g., in .symtab or .strtab sections). This maps human-readable names to memory addresses.
- **At Runtime:** The kernel is "symbol-blind." It only sees memory addresses and values. All variables reside in the **Process Address Space** (specifically in the Data segment for globals or the Stack for local variables), which are managed via the pagetable.
- **Debugging:** Tools like gdb or the xv6 serial console use the symbols from the binary file to map addresses back to names for the developer's convenience.

# CPU Scheduling

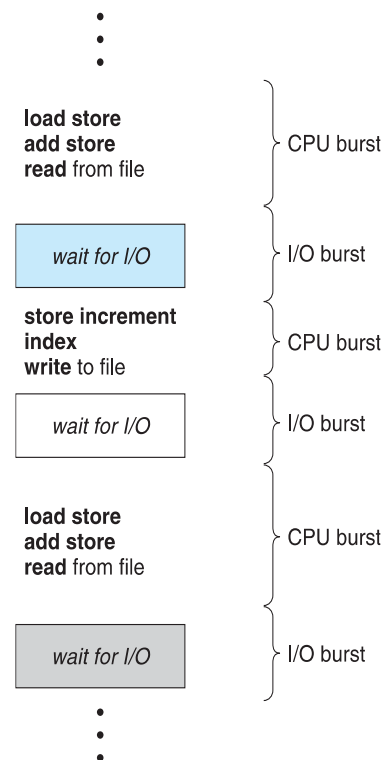
- CPU scheduling is a subset of process scheduling.
  - Process scheduling deals with the overall lifecycle and interactions of processes → processes may not always require the CPU. E.g. GPU rendering, I/O operation.
  - CPU scheduling specifically deals with deciding which process to execute next on the central processing unit (CPU). It involves selecting a process from the ready queue and allocating CPU time to that process.
  - However, all processes need the CPU at least once in their lifetime → for a process to be scheduled on the GPU, it must first have a CPU instruction requesting the transfer of data and GPU-specific instructions into the GPU.
- We will primarily be tackling CPU scheduling algorithms.

## Objective of CPU scheduling:

The primary goal of CPU scheduling is to maximize CPU utilization and throughput while minimizing waiting times and response times for processes.

## Background:

- Multiple processes compete for CPU time, and CPU scheduling determines which process to execute next.
- When the operating system switches from executing one process to another, it performs a context switch. This involves saving the current process's state and restoring the next process's state.
- Various scheduling algorithms exist to determine the order in which processes are executed. Common algorithms include First-Come, First-Served (FCFS), Shortest Job Next (SJN), Priority Scheduling, Round Robin, and Multilevel Queue Scheduling.
- A CPU can be on idle state such as the following case on the right. Program has “scattered” sections of I/O-related code. Whenever there’s an I/O code, the process must be removed from the CPU and move to the device queue (and gets executed in the I/O system program).
- A commercial OS will never have long CPU idle times even if the user is not seeing anything on-screen.



## Terms

- CPU – not the physical CPU hardware. A logical unit for executing a process.
- Ready queue – holds processes that should be executed in the CPU.
- Short-term scheduler - selects from among the processes in ready queue and allocates to the CPU.
- Long-term scheduler – decides which processes should go first to the short-term scheduler. NOTE: Not all OS have this.
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates

## Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue



- **Response time** - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

The **CPU Scheduling Problem** is formally defined as:

Given a set of processes  $P = \{P_1, P_2, \dots, P_N\}$  where  $P_i \in P$  contains  $\{A, C, X/\text{null}\}$  where:

- A = arrival time C = CPU time X = priority

and a **CPU scheduling algorithm**, find  $O$  which is a set of ordered  $P$  according to sequence of execution.

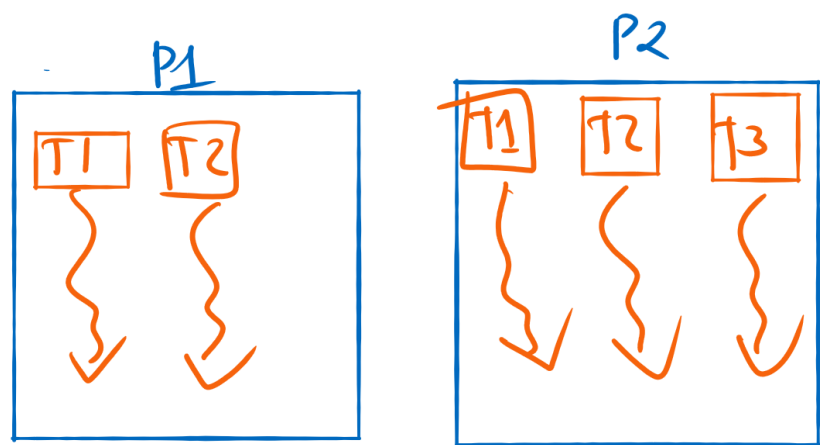
**CPU Scheduling Algorithm Criteria**

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

---

## INTRODUCTION TO THREADS

- Thread is a lightweight process. It allows us to parallelize certain logical operations.
- Contextualizing this to our marquee console, we can separate the marquee drawing from the keyboard polling.



- A thread is a lightweight process
- A sequence of instructions to execute in parallel with another thread.
- Since it is within a process, memory is shared.
- Multiple threads can access the same data structure, functions, etc.

**Hands-On Activity:** Create your first “Hello World” thread in C++.

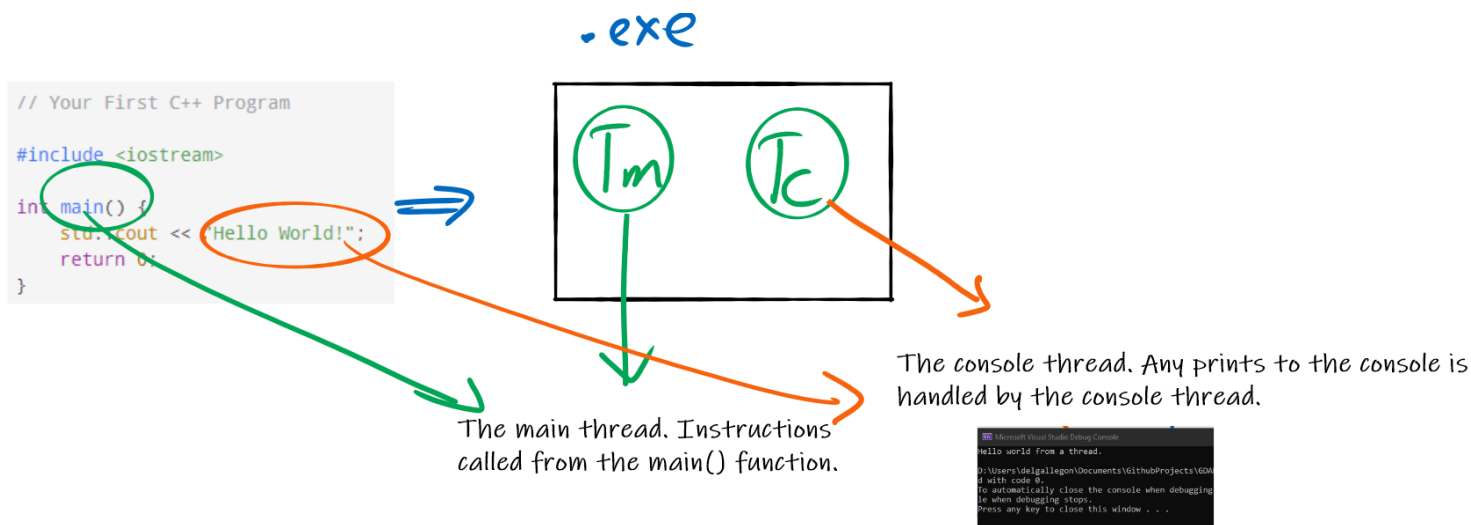
```
19 void testFunctionThread()
20 {
21 std::cout << "Hello world from a thread. " << std::endl;
22 }
23
24 int main() {
25 //createHWThreads();
26 std::thread myThread(testFunctionThread);
27 }
```

NOTE: Running the above program will trigger a runtime error. C++ has a guarding mechanism that should force the developer to call either `join()` or `detach()`.

```
24 int main() {
25 //createHWThreads();
26 std::thread myThread(testFunctionThread);
27 myThread.join();
28 }
```

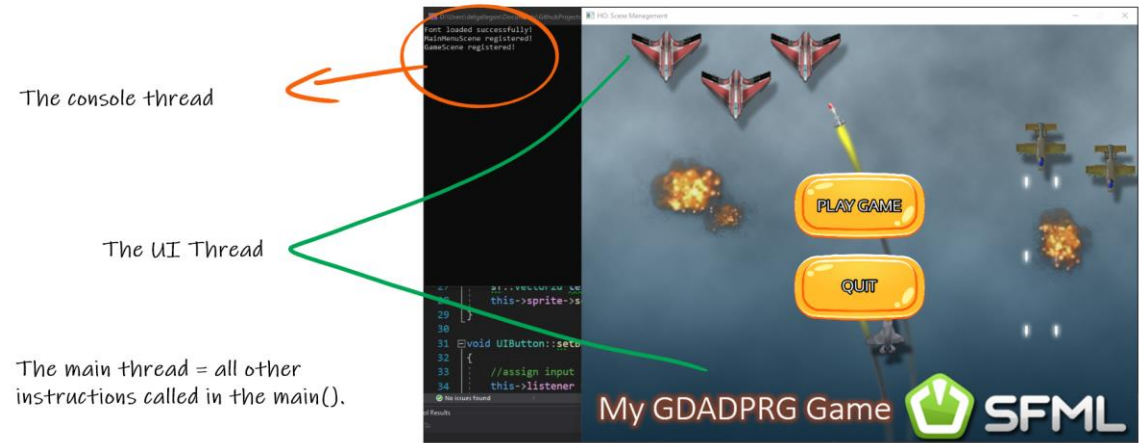
A Windows program - Default thread handling.

A default Windows program always has the following set of threads: **main thread**, **UI thread**. See illustration below.



The console thread persistently runs until the main program is closed.

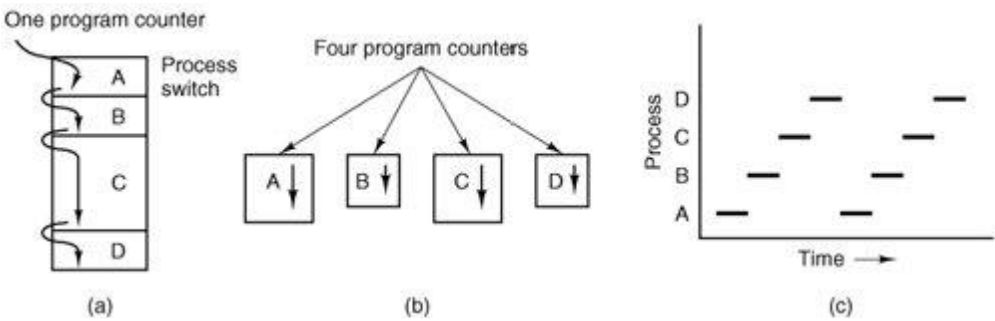
For programs that has a window screen, such as game applications, there are actually 3 threads by default.



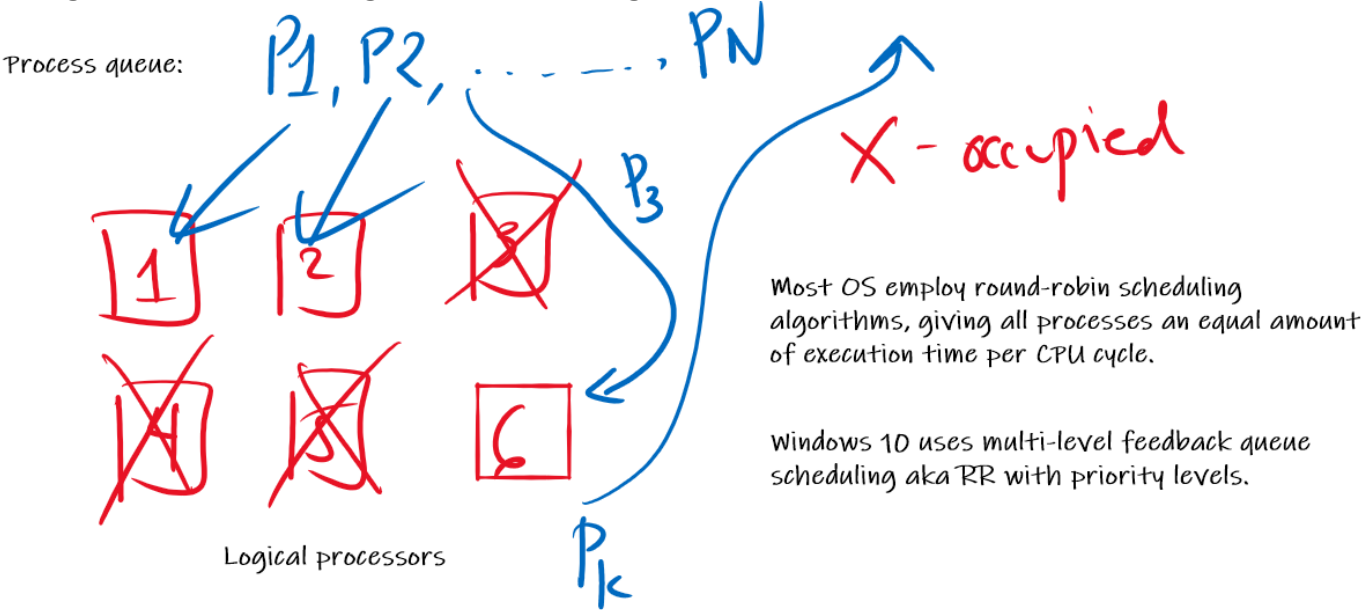
**NOTE:** All the spawned threads are independent from one another. Information is being passed to each thread through **message passing** (e.g. `std::cout`, `render/draw` frame in SFML/OPENGL).

**TODO:** Show a walkthrough of how a worker thread handles the marquee implementation: `MarqueeWorkerThread`.

Illustration of multiprocessing using a context-switching single-core processor vs multi-core processor



A diagram demonstrating CPU scheduling



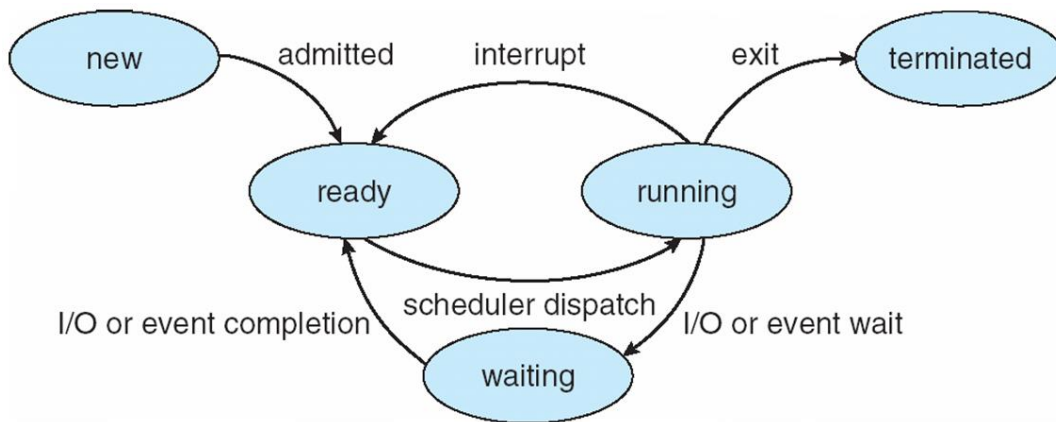
**Process transition**

- A program is in **passive** state when it is stored on disk (as an executable file).
- A program becomes a process when the executable file is loaded into memory (**active** state).
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can have several processes. E.g. consider a user opening a program multiple times.

**Process states**

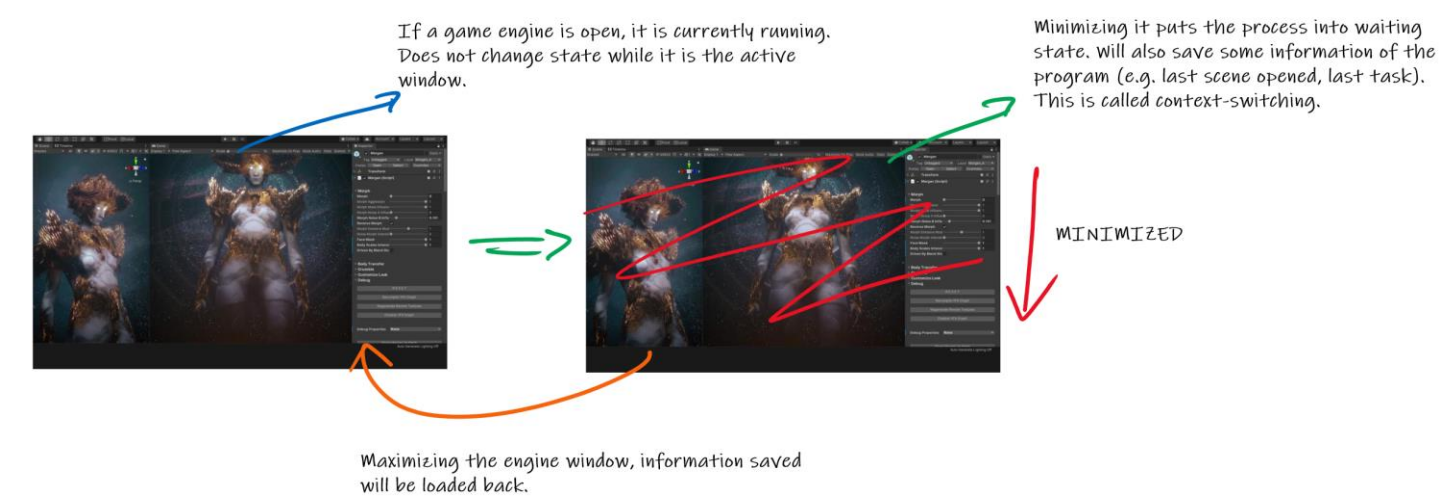
As a process executes, it changes **state**

- **new:** The process is being created
- **running:** Instructions are being executed
- **waiting:** The process is waiting for some event to occur
- **ready:** The process is waiting to be assigned to a processor
- **terminated:** The process has finished execution



Very similar to application lifecycle for mobile.

Processes can move to different states frequently. Consider this example:



OBSERVATION: Any active window implies that the process does not change state frequently.

| Background processes (147)           |      |          |
|--------------------------------------|------|----------|
| > Adobe Acrobat Update Service (...) | 0%   | 0.3 MB   |
| Adobe Collaboration Synchroni...     | 0%   | 1.5 MB   |
| Adobe Collaboration Synchroni...     | 0%   | 0.7 MB   |
| Adobe RdrcEF (32 bit)                | 0%   | 1.4 MB   |
| Adobe RdrcEF (32 bit)                | 0%   | 5.9 MB   |
| AlSuite3.exe (32 bit)                | 0%   | 0.4 MB   |
| > Antimalware Service Executable     | 2.1% | 193.9 MB |
| Application Frame Host               | 0.5% | 10.9 MB  |
| > AsSysCtrlService.exe (32 bit)      | 0%   | 0.3 MB   |
| > ASUS Com Service (32 bit)          | 0%   | 0.5 MB   |
| > ASUS Motherboard Fan Control ...   | 0%   | 0.7 MB   |
| AsusUpdateCheck.exe                  | 0%   | 0.6 MB   |
| > Calculator (2)                     | 0%   | 0.4 MB   |

However, background processes, such as services, move to various states frequently.

REASON: Foreground processes has more priority, a background process may stop execution at any time when additional resources are needed.

Hands-On Activity: Create your first “Hello World” thread in C++.

```
19 void testFunctionThread()
20 {
21 std::cout << "Hello world from a thread. " << std::endl;
22 }
23
24 int main() {
25 //createHWThreads();
26 std::thread myThread(testFunctionThread);
27 }
```

NOTE: Running the above program will trigger a runtime error. C++ has a guarding mechanism that should force the developer to call either join() or detach().

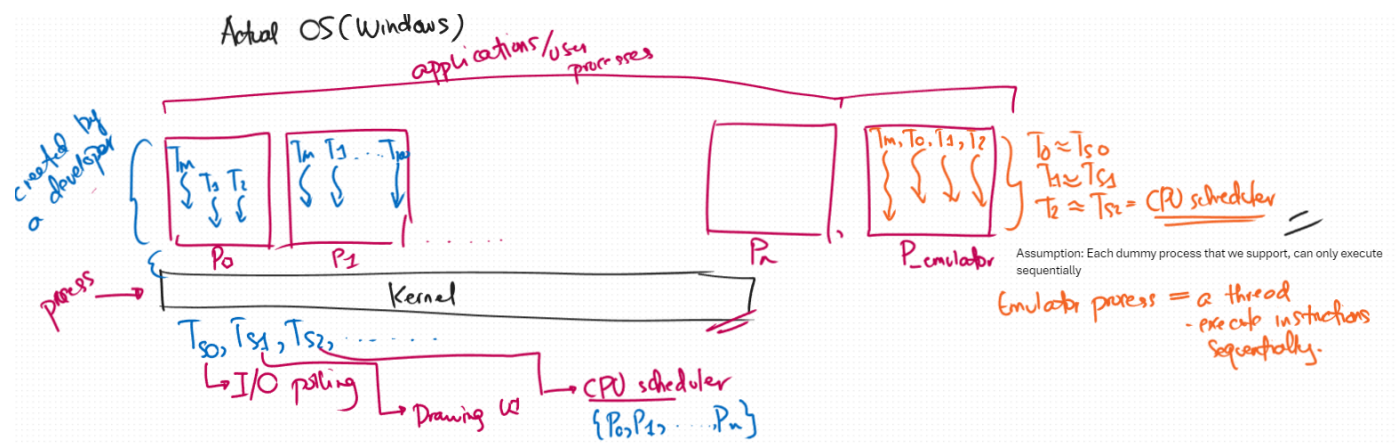
```
24 int main() {
25 //createHWThreads();
26 std::thread myThread(testFunctionThread);
27 myThread.join();
28 }
```

For our CSOPESY emulator, we will represent processes, as threads. Why?

- The application itself can be viewed as a box of your own PC, where we simulate how OS behaves.



- Processes use our imaginary resources → threads provide us easy access to these resources.

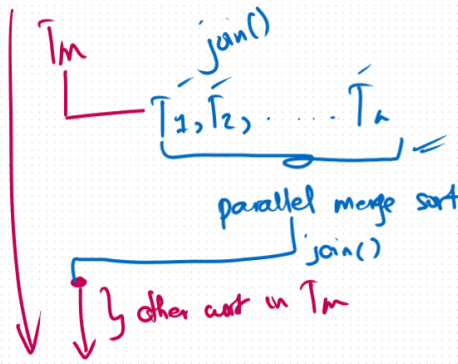
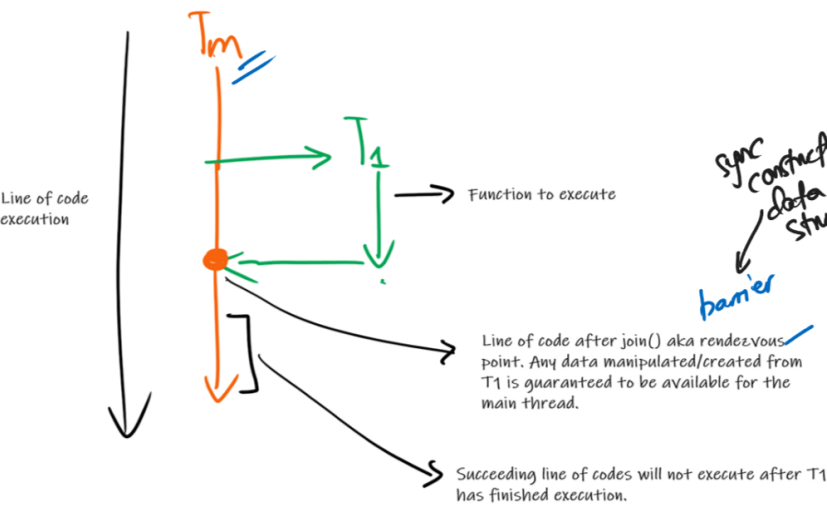


using `<thread.h>`

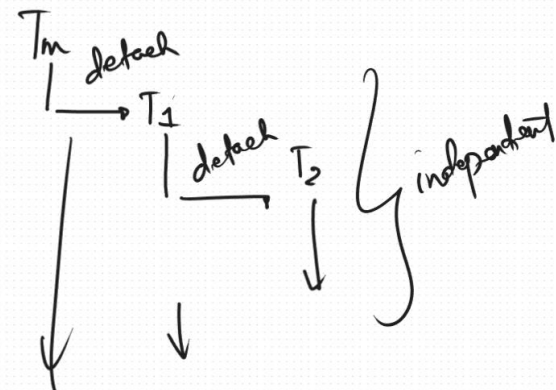
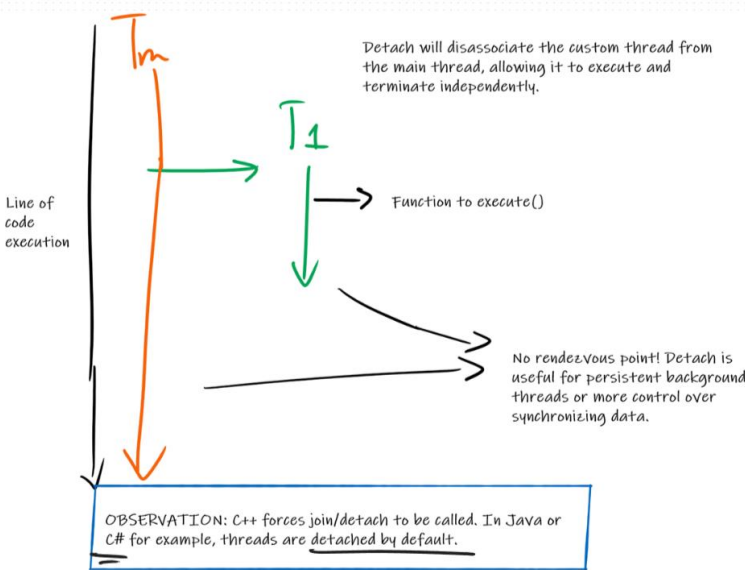
```
24 int main() {
25 //createHWThreads();
26 std::thread myThread(testFunctionThread);
27 myThread.join();
28 }
```

Based on C++'s compiler design, all threads must call either `join()` or `detach()`

Based on C++'s compiler design, all threads must call either `join()` or `detach()`.



Detach

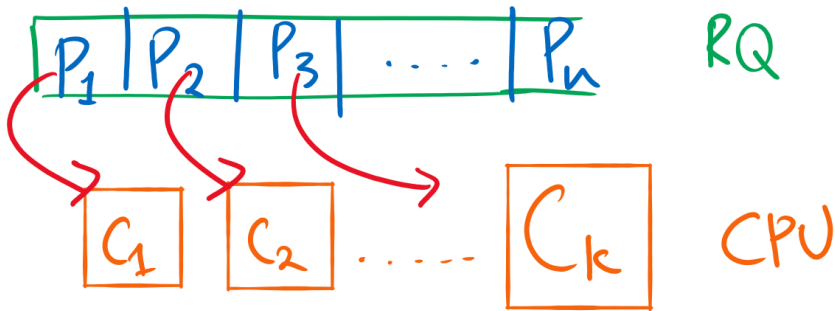




# Demonstration of CPU Scheduling: Shortest-Job-First (Pre-Emptive)

Shortest-job-first, by the name itself, selects processes with the shortest CPU burst for scheduling. Consider the following example:

The ready queue holds pending processes



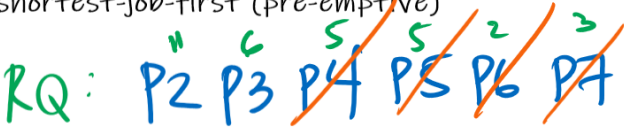
Each logical core can hold one process for execution.

When the process arrives at the ready queue

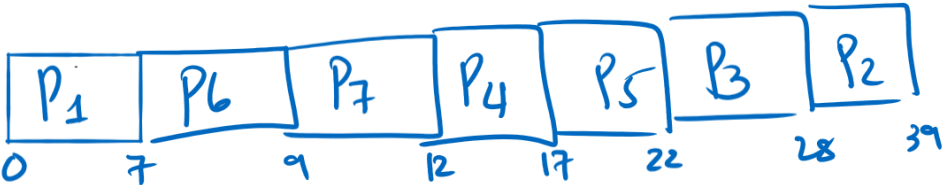
Total CPU time needed to finish it

| Process | Arrival Time | CPU Burst |
|---------|--------------|-----------|
| P1      | 0            | 7         |
| P2      | 1            | 11        |
| P3      | 2            | 6         |
| P4      | 3            | 5         |
| P5      | 4            | 5         |
| P6      | 6            | 2         |
| P7      | 7            | 3         |

Using shortest-job-first (pre-emptive)

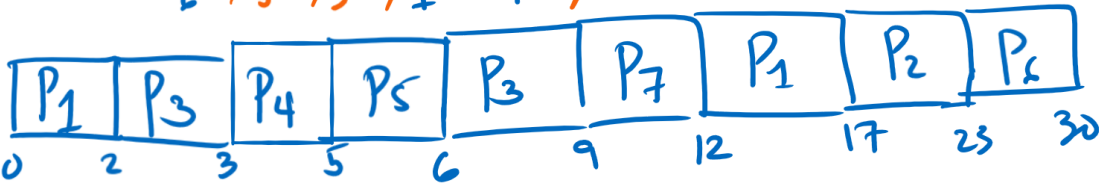


Gantt chart of CPU execution



To show how pre-emptive works

| Process | Arrival Time | CPU Burst       |
|---------|--------------|-----------------|
| P1      | 0            | 7               |
| P2      | 1            | <del>11</del> 6 |
| P3      | 2            | <del>6</del> 4  |
| P4      | 3            | <del>5</del> 2  |
| P5      | 4            | <del>5</del> 1  |
| P6      | 6            | <del>2</del> 7  |
| P7      | 7            | 3               |



# Algorithmic Overview of Pre-Emptive SJF

## Inputs

- List of processes inside the ready queue, R
- Given a set of processes  $P = \{P_1, P_2, \dots, P_N\}$  where  $P_i \in P$  contains  $\{A, C, X/\text{null}\}$  where:
  - A = arrival time
  - C = CPU time
  - X = priority

## Algorithm:

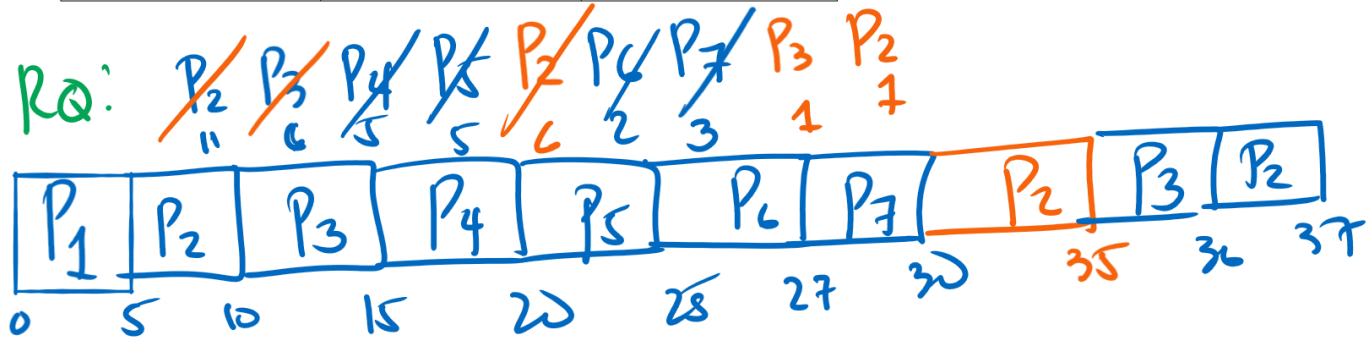
- For each CPU cycle, do the following:
  - Update R if there's any pending process to be scheduled.
  - If applicable, sort R where the first element satisfies  $P_0 = \min(C)$ .
  - Select  $P_{\text{candidate}}$ , such that  $P_{\text{candidate}} = \min(C)$
  - Execute  $P_{\text{candidate}}$ .  $P_{\text{candidate}}, C--$ ;

## Demonstration of CPU Scheduling: Round-Robin

Promotes fairness policy by cycling through all processes that arrive evenly. Each process receives a fair amount of CPU time for execution.

Using round-robin with CPU burst allocation of 5

| Process | Arrival Time | CPU Burst |
|---------|--------------|-----------|
| P1      | 0            | 7         |
| P2      | 1            | 11        |
| P3      | 2            | 6         |
| P4      | 3            | 5         |
| P5      | 4            | 5         |
| P6      | 6            | 2         |
| P7      | 7            | 3         |



IDEA: Promote fairness on all processes as much as possible

# Algorithmic Overview of Round-Robin

## Inputs

- List of processes inside the ready queue, R
- Time quantum slice, T.
- Given a set of processes  $P = \{P_1, P_2, \dots, P_N\}$  where  $P_i \in P$  contains  $\{A, C, X/\text{null}\}$  where:
  - A = arrival time
  - C = CPU cycles consumed. Starts with  $C = 0$ .
  - X = priority

## Algorithm:

- For each CPU cycle, do the following:

- a. Update R if there's any pending process to be scheduled.
- b. Select the first process in R (or randomly) to be the  $P_{candidate}$
- c. Execute  $P_{candidate} \cdot P_{candidate}, C++;$
- d. If  $P_{candidate}, C = T$ , then perform #b. Put  $P_{candidate}$  at the end of R. Otherwise, perform #c.

## Multi-Processor Scheduling

- When there are multiple cores/logical compute units present, two possible scheduling strategies could be:
  - All processes may be in a common ready queue
  - Each processor may have its own private queue of threads
- This approach is called **symmetric multiprocessing (SMP)**. The other approach is asymmetric multiprocessing.
  - CPU scheduling in a multiprocessor system has all scheduling decisions, I/O processing, and other system activities handled by a single processor – the master server.
  - This asymmetric multiprocessing is simple because only one core accesses the system data structures, reducing the need for data sharing. The downfall of this approach is the master server becomes a potential bottleneck where overall system performance may be reduced.

Illustration of SMP vs AMP is shown below:

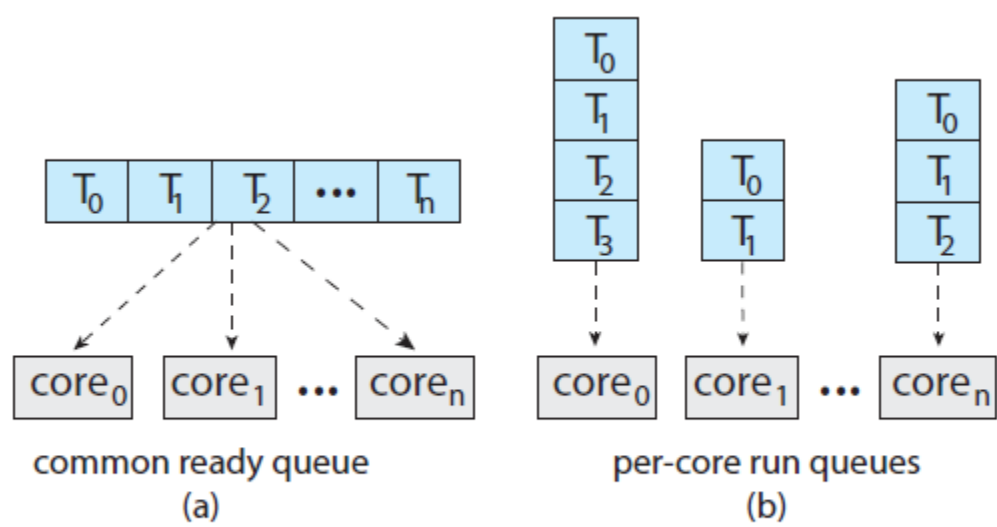


Figure 5.11 Organization of ready queues.

## Emulating a CPU scheduler

- We need the following: process representation, scheduler representation, and a way to debug/verify the correctness of our chosen scheduling algorithm.

The UI/frontend could first look like this:

```
X:\GithubProjects\CSOPESY_C + - 

* Displaying a scheduling console! Using scheduler: FCFScheduler *

Enter command: csopesy-smi
One bar means one command/remaining time.
PID:1 =====
PID:2 =====
PID:3 =====
PID:4 =====
PID:5 =====
PID:6 =====
PID:7 =====
PID:8 =====
PID:9 =====
PID:10 =====
PID:11 =====
PID:12 =====
PID:13 =====
PID:14 =====
PID:15 =====
PID:16 =====
PID:17 =====
PID:18 =====
PID:19 =====
PID:20 =====
PID:21 =====
PID:22 =====
PID:23 =====
```

OR

```

25 | _____
26 | / \ / \ / \ / \ / \ / \ / \ v /
27 | / \ / \ / \ / \ / \ / \ / \ /
28 | \ / \ / \ / \ / \ / \ / \ / \
29 | -----
30 | Welcome to CSOPESY Emulator!
31 |
32 | Developers:
33 | Del Gallego, Neil Patrick
34 |
35 | Last updated: 01-18-2024
36 |
37 | root:\> screen -ls
38 | CPU utilization: 100%
39 | Cores used: 4
40 | Cores available: 0
41 |
42 | -----
43 | Running processes:
44 | process05 (01/18/2024 09:15:22AM) Core: 0 1235 / 5876
45 | process06 (01/18/2024 09:17:22AM) Core: 1 3 / 5876
46 | process07 (01/18/2024 09:17:45AM) Core: 2 9 / 1000
47 | process08 (01/18/2024 09:18:58AM) Core: 3 12 / 80
48 |
49 | Finished processes:
50 | process01 (01/18/2024 09:00:21AM) Finished 5876 / 5876
51 | process02 (01/18/2024 09:00:22AM) Finished 5876 / 5876
52 | process03 (01/18/2024 09:00:42AM) Finished 1000 / 1000
53 | process04 (01/18/2024 09:00:53AM) Finished 80 / 80
54 | -----
55 |
56 | root:\> report-util
57 | root:\> Report generated at C:/csopesy-log.txt!
```

The user could always type “process-smi” to view the gantt chart. When all processes are finished, “csopesy-smi” should now display a summary of the ordering of processes executed and its **turnaround time**.

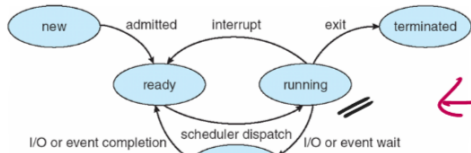
A representation of the process class:

```
556 //a template for the process
557 #include <iostream>
558 #include <string>
559 #include <cstdlib> // for rand() function
560
561 class Process {
562 private:
563 std::string name;
564 int id;
565 int totalInstructions;
566 int remainingInstructions;
567
568 public:
569 Process(const std::string& processName, int processId, int numInstructions)
570 : name(processName), id(processId), totalInstructions(numInstructions), remainingInstructions(numInstructions) {}
571
572 // Execute one instruction of the process
573 void executeInstruction() {
574 if (remainingInstructions > 0) {
575 std::cout << "Executing instruction for Process " << id << ": " << name << "\n";
576 remainingInstructions--;
577 } else {
578 std::cout << "Process " << id << ": " << name << " has already finished.\n";
579 }
580 }
581
582 // Get the remaining number of instructions
583 int getRemainingInstructions() const {
584 return remainingInstructions;
585 }
586
587 // Check if the process has finished
588 bool hasFinished() const {
589 return remainingInstructions == 0;
590 }
591 };
592
593 int main() {
594 // Create a sample process
595 Process myProcess("SampleProcess", 1, 10);
596
597 // Execute instructions until the process finishes
598 while (!myProcess.hasFinished()) {
599 myProcess.executeInstruction();
600 }
601
602 std::cout << "Process has finished!\n";
603
604 return 0;
605 }
```

Process representation

From Week 6 - Process Scheduling

- Process states
- As a process executes, it changes state
- new: The process is being created
  - running: Instructions are being executed
  - waiting: The process is waiting for some event to occur
  - ready: The process is waiting to be assigned to a processor
  - terminated: The process has finished execution



CPU scheduling decisions take place

Process table (ready)

Process table (running)

(finished)

```
42 private:
43 int pid;
44 String name;
45 typedef std::vector<std::shared_ptr<Command>> CommandList;
46 CommandList commandList;
47
48 int commandCounter;
49 int cpuCoreID = -1;
```

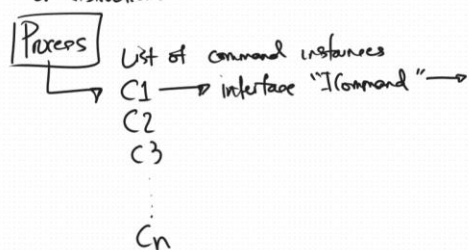


Process

int main()  
1  
2  
3  
4 } instructions/commands akin to machine code

program counter/current line of code  
Page P0, P1, ..., Pn  
0 1 2 ... K  
Pn-pe

Context: Process will have N number of commands or instructions



P<sub>0</sub> → "print" x100  
P<sub>1</sub> → "print" x1000

The process EXECUTES the list of commands in sequence.

Family of basic commands  
→ simplest instructions of a process  
"PRINT"  
"I/O"  
"SLEEP"  
"ADD"  
"DN"

In real-world/actual OS, the list of commands is created by your compiler --> discussed in a compiler theory course.

Source code → machine code

called by "CPU core worker"

```

30 void Process::executeCurrentCommand() const
31 {
32 this->commandList[this->commandCounter]-->execute();
33 }
34
35 void Process::moveToNextLine()
36 {
37 this->commandCounter++;
38 }

```

CPU scheduler (FCFS)

for RR, check the quantum slice remaining

Line 30  
Line 35



CPU core worker

CPU workers are individual threads

A template for a first-come-first-serve scheduler:

```

608 //a template for an FCFS scheduler
609 class FCFScheduler {
610 private:
611 int numCores;
612 std::vector<std::vector<Process>> processQueues; // One queue for each core
613
614 public:
615 FCFScheduler(int cores) : numCores(cores), processQueues(cores) {}
616
617 // Add a process to the scheduler
618 void addProcess(const Process& process, int core = 0) {
619 if (core >= 0 && core < numCores) {
620 processQueues[core].push_back(process);
621 } else {
622 std::cerr << "Invalid core specified for process addition.\n";
623 }
624 }
625
626 // Sort the process queues based on remaining instructions (FCFS)
627 void sortProcessQueues() {
628 for (auto& queue : processQueues) {
629 std::sort(queue.begin(), queue.end(), [](const Process& a, const Process& b) {
630 return a.getRemainingInstructions() > b.getRemainingInstructions();
631 });
632 }
633 }
634
635 // Run the scheduler
636 void runScheduler() {
637 while (!processQueues[0].empty()) {
638 for (int core = 0; core < numCores; ++core) {
639 if (!processQueues[core].empty()) {
640 Process currentProcess = processQueues[core].back();
641 processQueues[core].pop_back();
642
643 while (!currentProcess.hasFinished()) {
644 currentProcess.executeInstruction();
645 }
646
647 std::cout << "Process " << currentProcess.getRemainingInstructions() << " completed on Core " << core + 1 << ".\n";
648 }
649 }
650 }
651 }
652 };

```



The scheduler is designed to support multiple CPU cores that can be configured during compile-time.

Sample usage:

```
654 int main() {
655 // Seed the random number generator
656 std::srand(static_cast<unsigned>(std::time(0)));
657
658 // Assume the list of processes is given
659 std::vector<Process> processes;
660 processes.emplace_back("Process_1", 1, 10);
661 processes.emplace_back("Process_2", 2, 8);
662 processes.emplace_back("Process_3", 3, 15);
663
664 // Initialize FCFS Scheduler with 3 CPU cores
665 FCFSScheduler fcfsScheduler(3);
666
667 // Add processes to the scheduler
668 for (const auto& process : processes) {
669 fcfsScheduler.addProcess(process);
670 }
671
672 // Sort the process queues (FCFS)
673 fcfsScheduler.sortProcessQueues();
674
675 // Run the scheduler
676 fcfsScheduler.runScheduler();
677
678 return 0;
679 }
```

**TODO:** Discuss scheduler, process, and command implementations from the CSOPESY emulator.

**Activity:**

- Implement your schedulers in your emulator.
- Modify the scheduler such that the CPU core is actually a thread worker. If there are four cores, then processes get assigned to a "core", where its commands are executed.

Then, provide a function that can be called anytime from the main thread that writes down the CPU utilization with the following format:

|        | Util (%) | Num processes in queue |
|--------|----------|------------------------|
| CPU 1: | 100      | 4                      |
| CPU 2: | 100      | 5                      |
| CPU 3: | 100      | 2                      |
| CPU 4: | 100      | 6                      |

The command can be "cpu-util."

- Design your scheduler in such a way that it allows additional processes to be scheduled during run-time.
  - Behavior à at startup, create 50 random processes.
  - Then add a command "add-process-set", that creates 50 random processes and schedule them in the ready queue.
  - Note that "cpu-util" command can be used at any time.

**COMMON CPU SCHEDULING ALGORITHMS IN DETAIL**

- There are two commonly used scheduling algorithms: round-robin (RR) and shortest-job-first (SJF)

**Round-robin scheduling**

- Round-robin scheduling is a preemptive scheduling algorithm used in operating systems.



- It is designed for time-sharing systems, where each process is assigned a fixed time slot or quantum.
- Key Features:
  - Time Slicing:
    - Each process is assigned a fixed time slice or time quantum during which it can execute.
    - Once a process exhausts its time quantum, it is moved to the back of the ready queue.
  - Preemption:
    - If a process doesn't finish within its time slice, it is preempted, and the next process in the queue is given a chance to execute.
- Queue Structure:
  - Processes are organized in a circular queue (hence the name "round-robin").
  - The scheduler traverses the queue in a circular fashion, giving each process an opportunity to execute in turn.
- Advantages:
  - Fairness:
    - Provides fairness in CPU allocation as each process gets an equal share of CPU time.
  - Low Latency:
    - Results in low process response time due to quick switching between processes.
  - Simple Implementation:
    - Implementation is relatively simple compared to some other scheduling algorithms.
- Disadvantages:
  - Convoy Effect:
    - Can lead to the convoy effect, where short processes are delayed by long processes waiting in the queue.
  - Poor Performance for I/O-Bound Processes:
    - Inefficient for I/O-bound processes as they may lose their time slice while waiting for I/O.
- Parameter:
  - The main parameter is the time quantum, which determines the maximum time a process can run before being preempted.
- Example:
  - Consider three processes A, B, and C with time slices of 20 ms each.
  - The scheduler allocates CPU time to each process in a circular manner:  
A -> B -> C -> A -> B -> C -> ...

### Use Cases

- Round-robin scheduling is suitable for systems where fairness and quick response time are crucial, such as interactive systems and time-sharing environments.

### Dynamic Priority

- Some variants may adjust the priority of processes dynamically based on their behavior and resource needs.

### Windows OS scheduler

Windows operating systems use a scheduling algorithm that incorporates elements of round-robin scheduling. In Windows, the scheduler uses a priority-based approach involving process time-sharing. While it's not a strict round-robin algorithm, the idea of allocating time slices to different processes and regularly switching between them is present.

Windows uses a priority-based, preemptive scheduling algorithm where each process is assigned a priority level. The scheduler assigns time slices (quantum) to processes based on their priority. Higher-priority processes get more frequent access to the CPU. If a process doesn't finish within its time slice, it is moved to the back of the queue, allowing other processes to execute.

While the Windows scheduler is more sophisticated and dynamic than a simple round-robin scheduler, the concept of time-sharing and fairness among processes is still a key part of its design. The Windows scheduler considers factors like process priority, thread priority, and process states to make intelligent scheduling decisions.

### **Shortest-job-first**

- SJF is a non-preemptive or preemptive scheduling algorithm.
- Also known as Shortest Job Next (SJN) or Shortest Job Next.
- It selects the process that has the smallest total remaining processing time.
- Algorithm:
  - Processes are ordered by their total burst time (time required for execution).
  - The process with the shortest burst time is scheduled first.
  - If multiple processes have the same burst time, FCFS (First-Come-First-Serve) order is followed.
- Non-Preemptive SJF:
  - Once a process starts executing, it cannot be preempted until it completes.
  - Decisions are made only when a process completes or a new process arrives.
- Preemptive SJF:
  - Also known as Shortest Remaining Time First (SRTF).
  - A new process with a shorter burst time can preempt a currently running process.
  - This requires dynamic prioritization based on the remaining time.
- Advantages:
  - Minimizes waiting time and turnaround time.
  - Efficient for a system with predictable burst times.
- Disadvantages:
  - Requires knowledge of the total burst time, which is often not available in real-time. E.g. In Windows, all processes can run at any given time through user interactions à double-clicking on an icon.
  - Can lead to starvation for longer processes.
- Example Scenario:
  - Consider processes with burst times: P1(6), P2(8), P3(7), P4(3).
  - Non-Preemptive SJF would schedule: P4, P1, P3, P2.
- Note:
  - SJF is optimal for minimizing waiting time but requires accurate knowledge of process burst times, which may not be available in practice.

### **Multi-level queue scheduling**

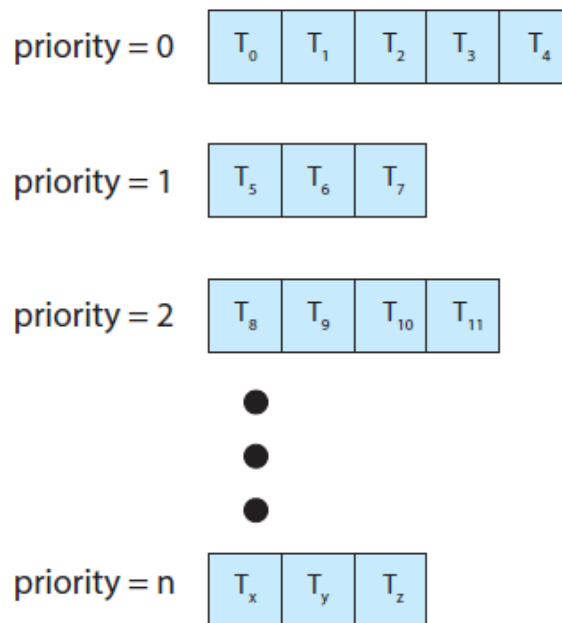


Figure 5.7 Separate queues for each priority.

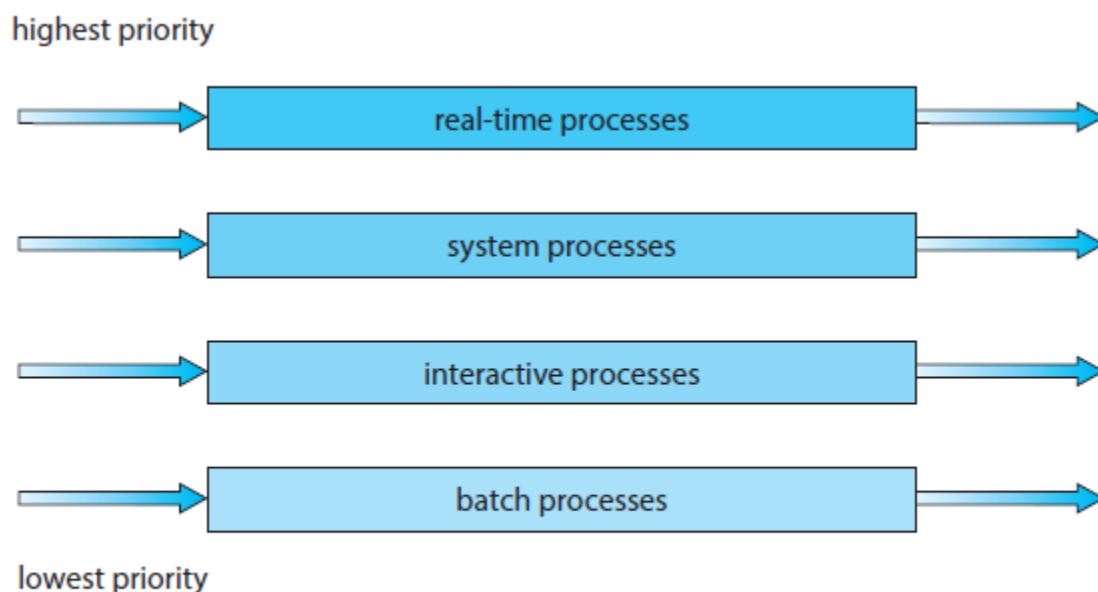


Figure 5.8 Multilevel queue scheduling.

- Overview:
  - Multi-Level Queue (MLQ) is a scheduling algorithm that divides processes into multiple queues based on priority.
  - Each queue can have its own scheduling algorithm.
  - Queues are assigned different priorities, and processes move between queues based on their characteristics.
- Key Components:
  - Multiple Queues:
    - Processes are assigned to one of several priority queues.
    - Each queue may have its own scheduling algorithm (e.g., FCFS, Round Robin).
  - Priority Assignment:
    - Queues are assigned different priorities.
    - Higher priority queues are processed before lower priority queues.
  - Queue Transition:
    - Processes can move between queues based on criteria (e.g., aging, priority adjustments).
    - Promotions and demotions based on process behavior.
- Algorithm Execution:

- **High-Priority Queue:**
  - Processes in the highest priority queue are scheduled first.
  - Processes may not be preempted within this queue.
- **Low-Priority Queue:**
  - Lower priority queues may use different scheduling algorithms.
  - Processes move to lower priority queues if they don't finish in the current queue.
- **Example Scenario:**
  - Consider three priority queues: High priority (Round Robin), Medium priority (FCFS), Low priority (SJF).
- **Advantages:**
  - Provides a way to manage processes with varying characteristics.
  - Better responsiveness for high-priority tasks.
- **Disadvantages:**
  - Requires careful tuning of priorities and transition rules.
  - May lead to starvation if not implemented properly.
- **Note:**
  - MLQ is a flexible approach that allows for different scheduling policies to be applied to different types of processes.

**More information about Windows scheduler**

Windows operating systems typically use a priority-based scheduling algorithm rather than a strict Multi-Level Queue (MLQ) scheduling approach. Windows uses a combination of priority levels, thread quantum, and a priority boost mechanism for thread scheduling.

In the Windows scheduling model:

- **Priority Levels:** Threads are assigned priority levels ranging from 0 to 31, with lower numbers indicating higher priority.
- **Quantum:** Each thread is given a time quantum during which it can execute. After the quantum expires, the thread may be moved to the back of the queue.
- **Priority Boost:** Windows incorporates a priority boost mechanism to prevent aging. If a thread waits too long for execution, it may receive a priority boost.

**Activities:**

- Implement some of your emulator's previously discussed scheduling algorithms, such as RR and SJF.
- Provide a design implementation for your multiprocessor scheduler following the diagrams provided below:

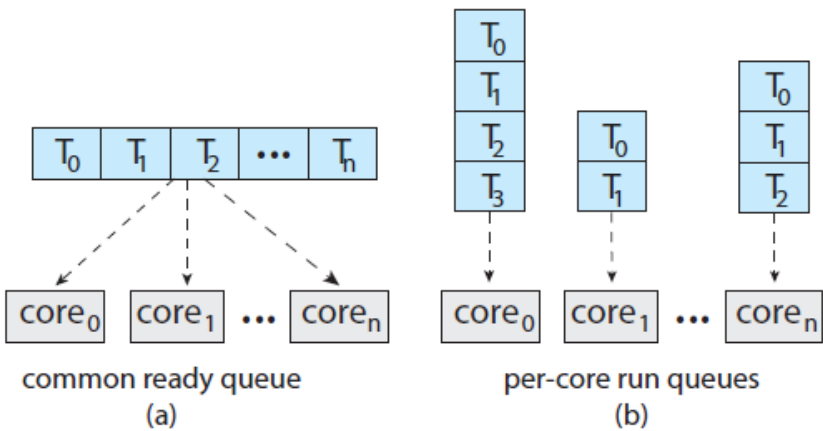


Figure 5.11 Organization of ready queues.

Design #1: There is only one ready queue.

Design #2: Ready queues are tied to CPU cores.